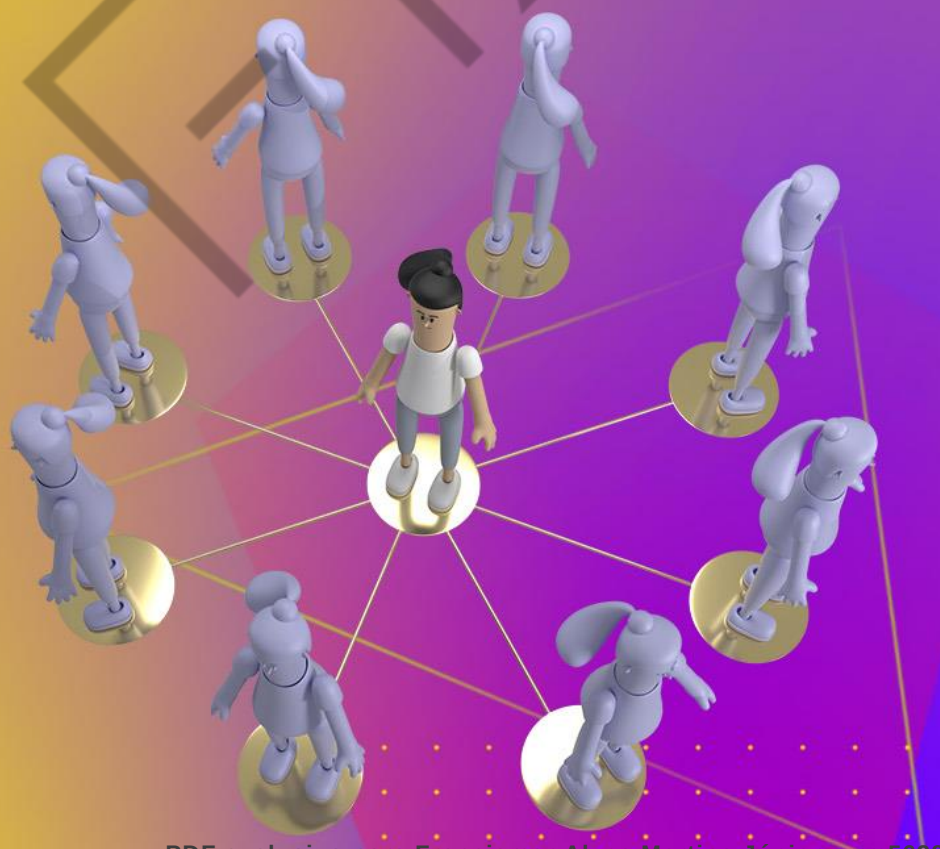
TRATORES DIGITAIS - HARDWARES PARA IA

MACHINE LEARNING SEM SUPERVISÃO: UMA JORNADA PELA DESCOBERTA DE DADOS



10

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Como dividir estes países em 4 grupos baseado apenas nas bandeiras? 6
Figura 2 - Exemplos de outliers.....7
Figura 3 - Intuição da clusterização (separar dados em grupos).....8
Figura 4 - Redução de dimensionalidade com PCA.....9
Figura 5 - Um exemplo de clusterização.....15
Figura 6 - Algumas medidas de distância 19
Figura 7 - Exemplo da influência dos dados sem normalização ou padronização 20
Figura 8 - Representação do método do cotovelo.....21
Figura 9 - Representação do método da silhueta.....22
Figura 10 - Diferença de clusterização de DBSCAN e K-Means.....23
Figura 11 - Formas de parametrização do DBSCAN 25
Figura 12 - Ilustração da clusterização hierárquica.....26
Figura 13 - Ilustração do método Mean Shift.....27
Figura 14 - Intuição do PCA sobre a base Iris Dataset 28
Figura 15 - Saída do código-fonte 1.....32
Figura 16 - Saída do código-fonte 2.....33
Figura 17 - Saída do código-fonte 3.....35
Figura 18 - Saída do código-fonte 4.....36
Figura 19 - Saída do código-fonte 5.....36
Figura 20 - Saída do código-fonte 6.....37
Figura 21 - Saída do código-fonte 7.....39
Figura 22 - Saída do código-fonte 8.....40
Figura 23 - Saída do código-fonte 9.....41
Figura 24 - Saída do código-fonte 10.....42
Figura 25 - Destaque para 2 clusteres formados de compradores do shopping 43
Figura 26 - Saída do código-fonte 11.....44
Figura 27 - Saída do código-fonte 12.....44
Figura 28 - Saída do código-fonte 13.....47
Figura 29 - Possíveis termos associados à “ética” 48
Figura 30 - Mensagem importante sobre ética em Machine Learning.....49
Figura 31 - Exemplo de discriminação algorítmica 1.....50
Figura 32 - Exemplo de discriminação algorítmica 2.....51
Figura 33 - Exemplo de discriminação algorítmica 3.....51
Figura 34 - Timeline do Blog do Tarcízio Silva sobre Ética em IA.....52
Figura 35 - Fontes de vieses em dados e Machine Learning: (a) O processo de geração de dados começa com a coleta de dados; (b) Um modelo é definido e otimizado nos dados de treinamento.....55

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Vantagens e desvantagens do DBSCAN 25
Quadro 2 - Problemas éticos e como evitá-los.....54

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Imports e leitura do dataset 1 32
Código-fonte 2 - Informações gerais da base 1.....33
Código-fonte 3 - Informações gerais da base 2.....34
Código-fonte 4 - Imports e leitura do dataset 2 35
Código-fonte 5 - Informações gerais da base 3.....36
Código-fonte 6 - Informações gerais da base 4.....36
Código-fonte 7 - Informações gerais da base 5.....38
Código-fonte 8 - Informações gerais da base 6.....39
Código-fonte 9 - Informações gerais da base 7.....40
Código-fonte 10 - Informações gerais da base 8.....42
Código-fonte 11 - Informações gerais da base 9.....44
Código-fonte 12 - Informações gerais da base 10.....44
Código-fonte 13 - Informações gerais da base 11.....46

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 MACHINE LEARNING SEM SUPERVISÃO: UMA JORNADA PELA DESCOBERTA DE DADOS 6
1.1 Introdução 6
1.1.1 Motivação 6
1.2 Formas de modelagem não supervisionada 8

2 APLICAÇÕES 10
2.1 Clusterização 10
2.2 Redução de dimensionalidade 12

3 AVALIANDO MODELOS NÃO SUPERVISIONADOS 13
3.1 O que é uma métrica? 13
3.2 Como avaliar modelos não supervisionados? 14
3.3 Métricas de qualidade da clusterização 14

4 PRINCIPAIS ALGORITMOS 17
4.1 Clusterização: K-Means 17
4.1.1 Algoritmo 17
4.1.2 Formas de parametrização 18
4.2 Clusterização: DBSCAN 23
4.2.1 Algoritmo 23
4.2.2 Formas de parametrização 24
4.3 Clusterização: Outras Abordagens 25
4.4 Redução de dimensões: PCA 27
4.4.1 Algoritmo 29
4.4.2 Formas de parametrização 29

5 HANDS ON: CONSTRUINDO MODELOS NÃO SUPERVISIONADOS 31
5.1 Sobre as bases de dados 31
5.2 Diferenciando K-means de DBSCAN 32
5.3 Agrupando consumidores e detectando padrões de compras 35
5.4 Eliminando outliers identificados pelo DBSCAN 41
5.5 Interpretando clusteres 43
5.6 Reduzindo dimensões com PCA 45

6 ÉTICA EM MACHINE LEARNING 48
6.1 O que é ética? 48
6.2 Qual é a importância da ética em Inteligência Artificial? 49
6.3 Exemplos de discriminação algorítmica 49
6.3.1 Câmeras da Nikon 49
6.3.2 Chatbot da Microsoft 50
6.3.3 FaceApp 51
6.3.4 Outros casos 52
6.4 Possíveis problemas éticos e como evitá-los 53

CONCLUSÃO 56

REFERÊNCIAS 57

GLOSSÁRIO 59

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 MACHINE LEARNING SEM SUPERVISÃO: UMA JORNADA PELA DESCOBERTA DE DADOS

1.1 Introdução

1.1.1 Motivação

Imagine que você tem a tarefa de dividir os países representados pelas bandeiras da figura a seguir em quatro grupos, utilizando apenas as informações visuais das bandeiras. Esse exercício levanta uma questão interessante: quais critérios você usaria para fazer essa divisão?

Figura 1 - Como dividir estes países em 4 grupos baseado apenas nas bandeiras?
Fonte: Dreamstime (2024)

Algumas opções para realizar essa divisão poderiam incluir as cores predominantes, como vermelho e azul, verde e branco, os formatos dos símbolos, ou até mesmo os elementos representados (como estrelas, listras ou emblemas). Cada critério escolhido visa agrupar as bandeiras destacando aspectos específicos e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

criando diferentes agrupamentos, o que revela como os padrões visuais podem levar a interpretações variadas.

Mas será que existe um critério “correto” ou “melhor”? A escolha do critério vai depender do objetivo da divisão. Você poderia escolher as cores caso o objetivo fosse identificar similaridades visuais, por exemplo; já se o objetivo for destacar traços culturais e históricos, basear-se nos símbolos seria um critério mais adequado. Esse exercício é uma introdução ao conceito de clusterização em Aprendizado de Máquina, onde o algoritmo precisa agrupar dados não rotulados com base em suas características. Não há um único critério “certo” em clusterização — o importante é escolher o critério que mais se aproxima do objetivo da análise.

No mundo dos dados, nem sempre temos rótulos (*labels*) claros para definir exatamente o que estamos procurando. Esse é o cenário perfeito para o **aprendizado não supervisionado**, onde o objetivo é explorar e encontrar padrões ocultos ou conexões desconhecidas nos dados. Em vez de se basear em respostas pré-definidas, o aprendizado não supervisionado nos permite agrupar dados similares, identificar estruturas complexas e fazer descobertas que poderiam passar despercebidas em uma análise tradicional.

Além de revelar agrupamentos, essa abordagem também é valiosa para a detecção de outliers — pontos de dados que destoam do comportamento geral —, como mostrados na figura “Exemplos de outliers”.

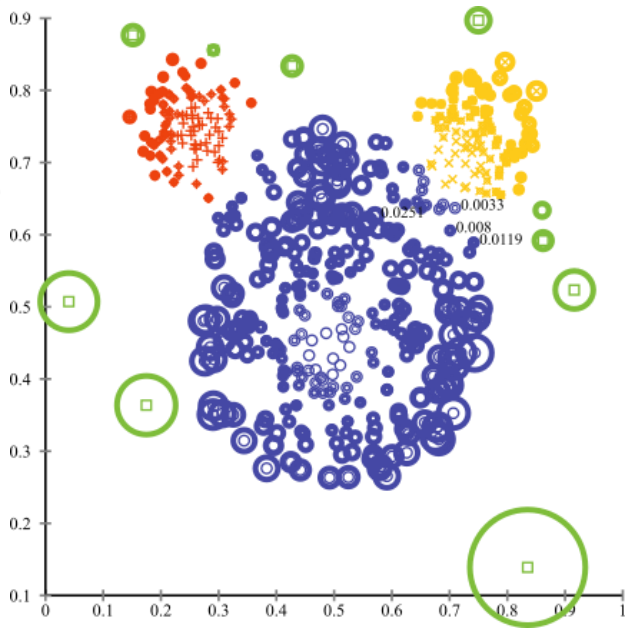


Figura 2 - Exemplos de outliers
Fonte: Cross Validated (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esses outliers podem representar anomalias importantes, como possíveis fraudes, erros ou até mesmo oportunidades inesperadas. Com o aprendizado não supervisionado, ganhamos uma ferramenta poderosa para investigar e entender os dados em profundidade, transformando o desconhecido em insights valiosos para a tomada de decisão.

1.2 Formas de modelagem não supervisionada

Os métodos de aprendizado não supervisionado são fundamentais quando lidamos com grandes volumes de dados não rotulados, uma situação comum em várias áreas, desde marketing até biomedicina. A **clusterização** nos permite identificar grupos naturais em conjuntos de dados, revelando padrões de comportamento ou perfis que não estavam evidentes. Por exemplo, em um banco de dados de clientes, a clusterização pode ajudar a segmentá-los em grupos de interesse, permitindo estratégias personalizadas para cada segmento. Esse agrupamento gera insights valiosos, mesmo quando não possuímos informações rotuladas previamente.

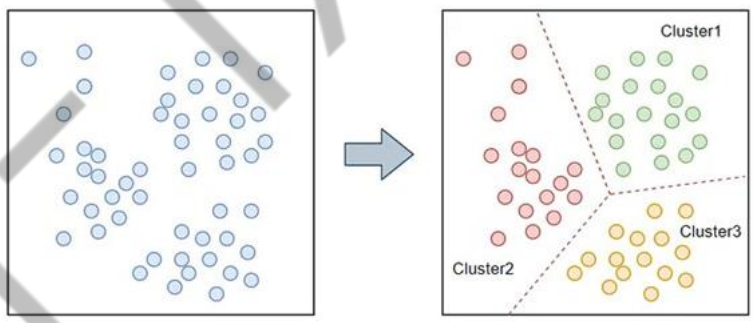


Figura 3 - Intuição da clusterização (separar dados em grupos)
Fonte: Data Geeks (2024)

Quanto à **redução de dimensionalidade**, ela é uma técnica crucial para simplificar dados complexos com muitas variáveis. Muitas vezes, um conjunto de dados possui dezenas ou centenas de variáveis, o que pode dificultar tanto o processamento quanto a interpretação. Métodos de redução de dimensionalidade, como a Análise de Componentes Principais (PCA), ajudam a transformar esses dados, mantendo as informações mais relevantes e descartando o “ruído”. Isso

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

permite que padrões subjacentes se tornem mais visíveis, além de facilitar o uso de algoritmos de aprendizado e melhorar o tempo de processamento, uma vantagem especialmente útil em Big Data.

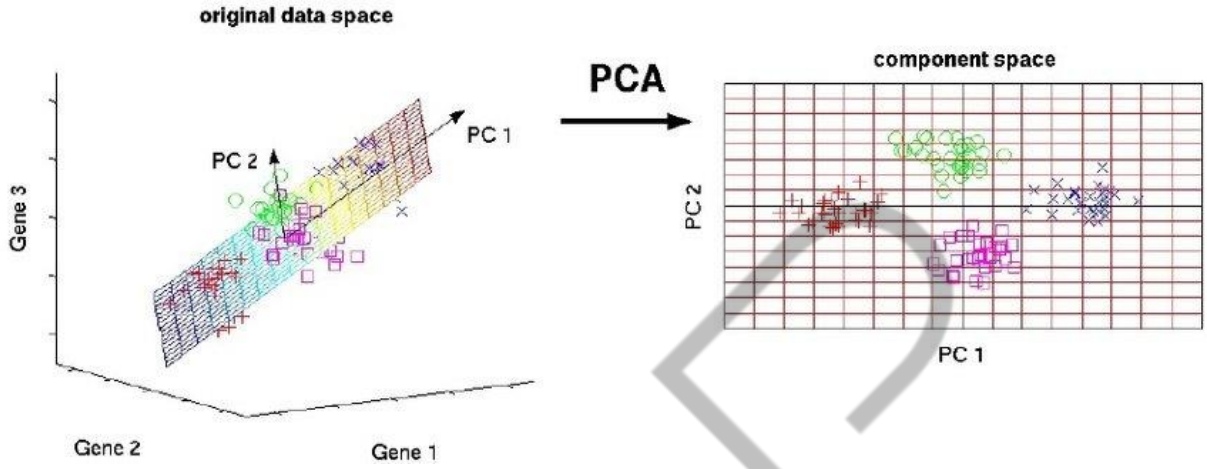


Figura 4 - Redução de dimensionalidade com PCA
Fonte: ResearchGate (2021)

A aplicação da redução de dimensionalidade vai além de otimizar o desempenho dos modelos, ela é uma ferramenta poderosa para visualização. Em áreas como genética, onde cada amostra pode ter milhares de características, técnicas como o PCA permitem visualizar os dados em duas ou três dimensões, revelando agrupamentos e anomalias que seriam invisíveis em um espaço de alta dimensionalidade. Outro exemplo é em análises de texto, onde métodos como a Singular Value Decomposition (SVD) reduzem o número de características, transformando textos em representações mais gerenciáveis e identificando relações semânticas entre palavras.

Portanto, a motivação para utilizar aprendizado não supervisionado é clara: ele proporciona uma maneira de explorar dados desconhecidos, identificar padrões, simplificar informações e até mesmo encontrar outliers. Em vez de depender de rótulos ou categorias pré-definidas, esses métodos permitem que o próprio dado “converse” conosco, guiando descobertas e revelando insights ocultos. Seja agrupando dados similares ou reduzindo a complexidade dimensional, o aprendizado não supervisionado é uma ferramenta indispensável para transformar grandes volumes de dados em informações acionáveis.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 APLICAÇÕES

2.1 Clusterização

A clusterização tem uma ampla gama de aplicações práticas em setores variados, desde o varejo até o setor de energia. Em cada um desses contextos, a técnica é utilizada para segmentar dados e encontrar padrões ocultos, facilitando a tomada de decisão e a personalização de estratégias.

No **varejo**, por exemplo, onde há uma vasta quantidade de dados sobre clientes e produtos, a clusterização permite identificar grupos de consumidores com comportamentos semelhantes, otimizando campanhas de marketing e aumentando a satisfação do cliente.

- **Segmentação de clientes:** agrupando consumidores com padrões de compra semelhantes, facilitando campanhas personalizadas.
- **Otimização de inventário:** analisando padrões de demanda para ajustar estoques de acordo com as preferências locais.

Na **mineração**, onde o objetivo é maximizar a eficiência e reduzir custos, a clusterização pode ajudar a identificar padrões de solo e minérios, facilitando operações e decisões de extração mais precisas.

- **Categorização de áreas de mineração:** agrupando zonas com características de solo e composição semelhantes, otimizando a alocação de recursos.
- **Identificação de anomalias geológicas:** destacando áreas com potencial para novas descobertas ou riscos.
- **Manutenção preditiva de máquinas:** agrupando máquinas com padrões de uso similares para otimizar o planejamento de manutenção.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No setor **agropecuário**, onde o monitoramento do solo, do clima e das plantações é essencial, a clusterização pode otimizar o manejo de lavouras, auxiliando no aumento da produtividade e sustentabilidade.

- **Agrupamento de safras:** identificando áreas com características similares para adaptar práticas agrícolas.
- **Segmentação de zonas climáticas:** agrupando áreas por condições meteorológicas para ajustes no cultivo.
- **Monitoramento de saúde animal:** identificando padrões de comportamento para prevenir doenças no gado.

Na **saúde**, um setor que lida com grandes quantidades de dados de pacientes, exames e diagnósticos, a clusterização permite segmentar os perfis de pacientes e identificar padrões de risco.

- **Agrupamento de pacientes:** identificando perfis de risco para personalizar tratamentos e prever complicações.
- **Análise de padrões de doenças:** agrupando sintomas e diagnósticos para ajudar na descoberta de novas patologias.
- **Pesquisa genética:** identificando grupos de genes com comportamentos similares para estudos de predisposição a doenças.

No setor de **energia**, onde a gestão de recursos e a previsão de demanda são essenciais, a clusterização é utilizada para otimizar operações e monitorar o consumo.

- **Segmentação de perfis de consumo:** agrupando consumidores com padrões de uso semelhantes para tarifas diferenciadas.
- **Agrupamento de sensores e medidores:** detectando anomalias e garantindo a eficiência do sistema.
- **Planejamento de redes inteligentes:** identificando padrões de demanda para melhorar a distribuição de energia.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Outras áreas, como finanças e educação, também utilizam a clusterização. Em **finanças**, para segmentar perfis de investimento e detecção de fraudes; na **educação**, para agrupar alunos com padrões de aprendizado similares, permitindo abordagens mais individualizadas. Cada setor se beneficia da clusterização ao tornar os dados mais organizados e acionáveis, revelando insights que apoiam estratégias eficazes e decisões informadas.

2.2 Redução de dimensionalidade

A redução de dimensionalidade é uma técnica essencial em vários setores para simplificar grandes volumes de dados, melhorar a performance de modelos e facilitar a interpretação. Em cada contexto, seu uso permite que informações relevantes sejam extraídas sem o excesso de variáveis, tornando os dados mais manejáveis. No setor de **varejo**, onde existem diversos dados de clientes e produtos, a redução de dimensionalidade ajuda a identificar as características mais relevantes para análise, eliminando redundâncias e otimizando os modelos.

Na área de **saúde**, onde o volume de dados clínicos, genéticos e de exames é vasto, a redução de dimensionalidade é aplicada para focar nas variáveis mais significativas para diagnóstico e análise de padrões. No setor **financeiro**, que lida com grande quantidade de informações econômicas e comportamentais, a redução de dimensionalidade permite identificar fatores-chave que influenciam o mercado e o comportamento do consumidor.

No setor de **manufatura**, a redução de dimensionalidade auxilia no monitoramento e análise de processos industriais que envolvem inúmeros sensores e variáveis. No setor de **energia**, a redução de dimensionalidade também ajuda a simplificar o monitoramento do consumo e das operações.

Assim, a redução de dimensionalidade não só facilita a análise e a visualização dos dados, mas também otimiza o desempenho de modelos e processos em uma ampla gama de aplicações, fornecendo informações claras e acionáveis em meio a grandes volumes de variáveis.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 AVALIANDO MODELOS NÃO SUPERVISIONADOS

3.1 O que é uma métrica?

Uma métrica é uma ferramenta utilizada para quantificar e avaliar o desempenho de um modelo de Aprendizado de Máquina. Ela representa uma medida numérica que traduz a precisão, a eficiência ou a relevância dos resultados produzidos pelo modelo em relação ao objetivo da análise. Em aprendizado supervisionado, onde temos rótulos pré-definidos (ou *labels*) para cada exemplo no conjunto de dados, as métricas são usadas para comparar as previsões do modelo com esses rótulos conhecidos, facilitando a mensuração de acertos e erros. Assim, métricas como acurácia, precisão, recall e F1-Score são amplamente utilizadas para avaliar o quão bem o modelo consegue classificar ou prever dados rotulados.

Essas métricas supervisionadas analisam principalmente a capacidade do modelo de “aprender” a partir de exemplos passados para prever corretamente novos dados. Por exemplo, em um problema de classificação binária, a acurácia mede a proporção de previsões corretas em relação ao total de previsões, enquanto o F1-Score pondera acertos e erros de acordo com a relevância, equilibrando precisão e recall. No caso de problemas de regressão, métricas como o Erro Absoluto Médio (MAE) e o Erro Quadrático Médio (MSE) quantificam o quão próximo a previsão numérica do modelo está dos valores reais.

Em modelos não supervisionados, no entanto, a avaliação é mais complexa, pois não há rótulos para orientar a análise. Em vez de comparar previsões com valores conhecidos, as métricas em aprendizado não supervisionado avaliam a estrutura e a qualidade dos agrupamentos ou a representação dos dados em dimensões reduzidas. O objetivo aqui é medir a coerência e a separação entre grupos (no caso de clusterização) ou a preservação da variação original (no caso de redução de dimensionalidade). Essas métricas são úteis pois servem como guias para avaliar se o modelo está capturando padrões relevantes e úteis, mesmo sem um “gabarito” com as respostas corretas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.2 Como avaliar modelos não supervisionados?

Avaliar modelos não supervisionados é uma tarefa desafiadora, pois, ao contrário do aprendizado supervisionado, aqui não temos uma “verdade terrestre” — ou seja, rótulos predefinidos ou valores conhecidos para comparar diretamente os resultados do modelo. Em vez de medir o quão bem o modelo acertou uma previsão específica, precisamos avaliar se ele identificou padrões ou relações relevantes nos dados, o que é um conceito mais abstrato e interpretativo.

Por natureza, essa avaliação é **intrinsecamente subjetiva**. Ou seja, não existem respostas certas ou erradas claramente definidas em modelos não supervisionados; o que temos são agrupamentos ou representações dos dados que podem ou não ser úteis, dependendo do contexto e do objetivo do estudo. Portanto, é comum que o processo de avaliação exija uma interação constante e uma análise crítica por parte dos analistas, que precisam interpretar os resultados para decidir se fazem sentido dentro do domínio específico de aplicação.

A interação e a inteligência humana são, assim, partes essenciais na avaliação de modelos não supervisionados. Embora existam métricas que auxiliem a medir a consistência interna dos grupos ou a separação entre eles, tais métricas não capturam sozinhas a relevância ou a aplicabilidade dos resultados no mundo real. Em resumo, é responsabilidade do especialista decidir se o modelo forneceu insights úteis ou se o agrupamento revelou informações acionáveis, estabelecendo um equilíbrio entre as medidas objetivas e a compreensão contextual do problema.

3.3 Métricas de qualidade da clusterização

As métricas de **coesão** e **separação** são comumente utilizadas para avaliar a qualidade de um modelo de clusterização, fornecendo uma forma de medir o quão bem os grupos formados pelo modelo representam os dados. Estas métricas, mesmo que objetivas, oferecem uma visão quantitativa da organização dos dados dentro e entre os clusters.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura a seguir mostra um exemplo de clusterização. Em uma boa clusterização, os clusteres formados são **bem definidos** (coesos) e **distantes** (separados).

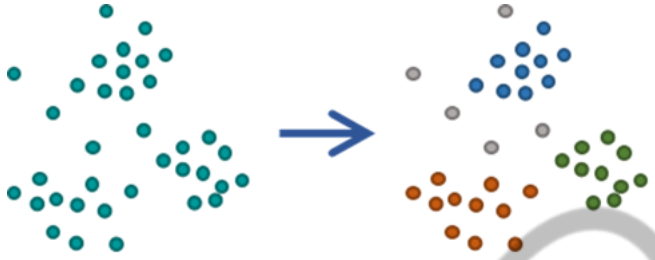


Figura 5 - Um exemplo de clusterização
Fonte: Blog.esrij (2024)

Vamos entender como funciona cada um desses conceitos. A **coesão** refere-se a quão próximos ou similares os pontos de dados estão dentro de um mesmo cluster. A ideia é que, em um bom agrupamento, os elementos de um cluster devem estar o mais próximo possível uns dos outros, indicando que o cluster está bem definido e os dados são internamente consistentes. Quanto menor a distância entre os pontos dentro de um cluster, maior a coesão. Existem algumas formas de medir a coesão, como:

- **Soma das distâncias do centro aos pontos do cluster:** calcula a soma das distâncias entre o centroide do cluster e cada ponto que pertence a esse cluster. Essa medida é usada para avaliar o quanto os pontos estão concentrados ao redor do centroide — quanto menor o valor, maior a coesão do cluster.
- **Maior distância entre dois pontos do cluster:** mede a distância máxima entre dois pontos dentro do cluster. Isso dá uma ideia do “diâmetro” do cluster, indicando até onde os pontos se espalham. Uma distância menor sugere que o cluster é mais compacto.
- **Média das distâncias entre pontos do cluster:** calcula a distância média entre todos os pares de pontos no cluster. Essa medida indica a proximidade média entre os pontos de um mesmo cluster. Quanto menor a média, mais próximos estão os pontos entre si, refletindo uma boa coesão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por outro lado, a **separação** refere-se à distância entre clusters diferentes. Em um bom modelo de clusterização, os clusters devem ser bem separados, ou seja, os pontos de um cluster devem estar bem afastados dos pontos dos outros clusters. Isso indica que os grupos formados são distintos e que o modelo está identificando diferenças significativas entre os dados. Existem algumas formas de medir a separação, como:

- **Distância entre os centros dos grupos:** essa medida calcula a distância entre os centroides (centros) dos clusters *i* e *j*. Uma maior distância entre os centros dos clusters indica que os grupos estão bem separados, o que é desejável para a criação de clusters bem definidos.
- **Menor distância entre os pontos de *i* e de *j*:** mede a menor distância entre qualquer par de pontos entre os clusters *i* e *j*. Essa medida é importante para identificar se há pontos muito próximos entre clusters diferentes, o que pode indicar sobreposição e comprometer a separação entre os grupos.
- **Média das distâncias entre pontos de *i* e de *j*:** calcula a distância média entre todos os pares de pontos pertencentes aos clusters *i* e *j*. Uma média maior indica uma separação mais clara entre os clusters, enquanto uma média menor pode indicar que os clusters estão mais próximos entre si.

Essas métricas são fundamentais para garantir que os clusters formados são internamente consistentes (alta coesão) e suficientemente diferentes uns dos outros (alta separação), proporcionando uma visão mais clara da qualidade do modelo de clusterização.

Felizmente, não é necessário calcular e interpretar tais métricas, pois elas são calculadas internamente no treinamento dos algoritmos de clusterização, e são utilizadas para diferenciar a qualidade de clusters distintos formados por processos aleatórios ou inicializações distintas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 PRINCIPAIS ALGORITMOS

4.1 Clusterização: K-Means

O K-Means é um método de **clusterização** não supervisionado que visa dividir um conjunto de dados em *K* grupos (clusters) distintos, onde cada ponto é associado ao cluster com o centroide mais próximo. A intuição por trás do K-Means é simples: agrupar pontos que estejam próximos uns dos outros, e separar grupos distintos para maximizar a coesão dentro de cada cluster e a separação entre clusters. O K-Means busca minimizar a variabilidade interna de cada cluster, agrupando dados similares e criando uma divisão natural dos dados com base na proximidade espacial.

4.1.1 Algoritmo

O algoritmo K-Means segue os seguintes passos principais:

1. **Inicialização:** escolher K pontos aleatoriamente no espaço de dados como centroides iniciais.
2. **Atribuição de clusters:** atribuir cada ponto de dados ao cluster cujo centroide está mais próximo, calculando a distância (geralmente a distância euclidiana).
3. **Recalcular centroides:** atualizar a posição de cada centroide como a média dos pontos atribuídos ao respectivo cluster.
4. **Iteração:** repetir os passos de atribuição e recalculação até que os centroides não mudem significativamente ou até atingir um número máximo de iterações. O objetivo é que a soma das distâncias entre os pontos e seus respectivos centroides seja minimizada.

Esse processo é iterativo e converge quando os clusters se estabilizam, isto é, quando não há mais mudanças nas atribuições de pontos aos clusters.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.1.2 Formas de parametrização

O K-Means possui alguns parâmetros importantes que influenciam seu desempenho e os resultados: o método de inicialização dos centroides; o algoritmo de distância; o número de clusteres; e o critério de convergência. A seguir, vamos analisar como as escolhas impactam os resultados.

Método de inicialização dos centroides

O ponto de partida dos centroides pode afetar o resultado, pois o K-Means é sensível a essa escolha. Uma inicialização ruim pode levar a um mínimo local, gerando resultados subótimos. As principais escolhas são:

- **Aleatória:** se queremos montar 4 clusteres, escolhemos 4 pontos ao acaso para serem os centroides. É a inicialização mais fácil, porém pode não gerar clusteres bem separados.
- **Pré-determinada:** se queremos montar 4 clusteres, mas conhecemos 4 pontos que representam bem as tendências ou padrões a serem explorados, podemos escolher tais pontos como centroides, e forçar o algoritmo a encontrar pontos similares aos escolhidos. Como resultado, temos clusteres com boa coesão.
- **Maximização de distâncias:** o método K-Means++ é uma técnica de inicialização que melhora a robustez, escolhendo pontos de forma a maximizar a distância inicial entre eles. Isso garante a geração de clusteres bem separados.

Algoritmo de distância

O ponto central de funcionamento do K-Means é a distância entre os pontos. Portanto, é imprescindível uma forma de mensurarmos tal distância. Existem diversas estratégias para mensurar a distância de um ponto “A” a um ponto “B” no espaço, algumas delas ilustradas na figura “Algumas medidas de distância”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

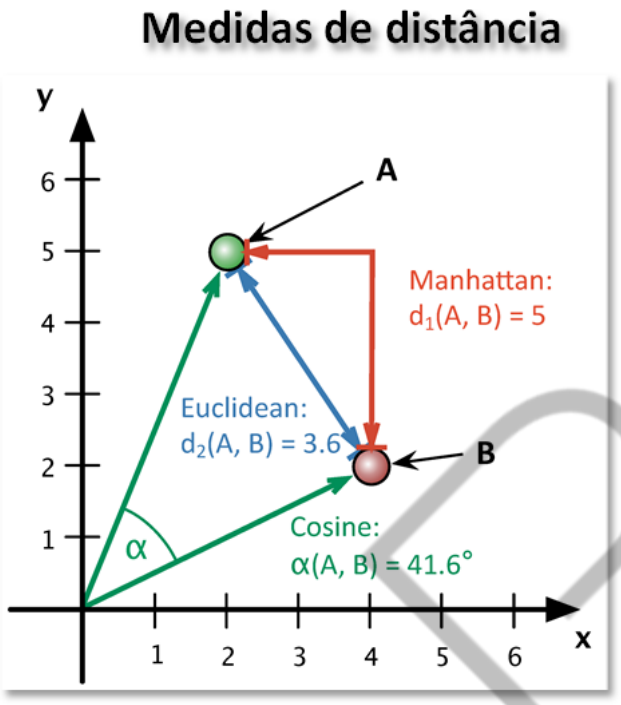


Figura 6 - Algumas medidas de distância
Fonte: Medium (2020)

O racional de funcionamento destas medidas é:

- **Distância Euclidiana:** a métrica mais comum para K-Means, mede a distância em linha reta entre dois pontos no espaço.
- **Distância de Manhattan:** soma das distâncias absolutas entre as coordenadas dos pontos, adequada para dados com componentes independentes.
- **Distância de Coseno:** mede a similaridade com base no ângulo entre os vetores, sendo útil quando a magnitude dos dados é irrelevante.

Vale ressaltar também que todo método baseado em distância exige a normalização/padronização das variáveis como etapa de pré-processamento dos dados. Isso facilita a convergência e impede a geração de hipóteses falsas sobre a importância de cada feature dentro do modelo. Veja como exemplo a figura “Exemplo da influência dos dados sem normalização ou padronização”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

ATRIBUTO	PESSOA A	PESSOA B
Gênero	Masculino	Feminino
Idade	23	40
Anos na residência atual	2	10
Status residencial (1=Proprietário; 2=Inquilino; 3=Outro)	2	1
Renda (R$)	20.000	90.000

Figura 7 - Exemplo da influência dos dados sem normalização ou padronização
Fonte: Elaborado pelo autor (2024)

Se a ideia é compararmos a “distância” entre as pessoas “A” e “B”, para verificarmos se são parecidas ou não, é imprescindível normalizar as features colocando-as na mesma escala. Do contrário, a diferença numérica entre renda será muito maior do que idade. Isso pode dar a falsa informação para o modelo que renda é mais relevante para diferenciar as pessoas do que idade.

Talvez essa diferença faça sentido em uma análise de aumento de crédito imobiliário, mas pode não ser realista no score de risco para uma doença. Portanto, escalar as variáveis, seja por normalização ou padronização, é um pré-processamento mandatório em qualquer modelo baseado em distância de pontos.

- Número de clusteres:

Definir o número de clusters é o principal desafio. Métodos como o “Elbow Method” ou o Índice de Silhueta ajudam a escolher um valor apropriado, mas essa escolha depende do conhecimento do problema e da estrutura dos dados.

- Método do cotovelo:

O método do cotovelo é uma técnica gráfica utilizada para determinar o número ideal de clusters *K* no K-Means. A ideia é calcular a soma das distâncias quadradas internas (Within-Cluster Sum of Squares, conhecido também pela sigla WCSS) para diferentes valores de *K* e plotá-los em um gráfico, como ilustrado na figura “Representação do método do cotovelo”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

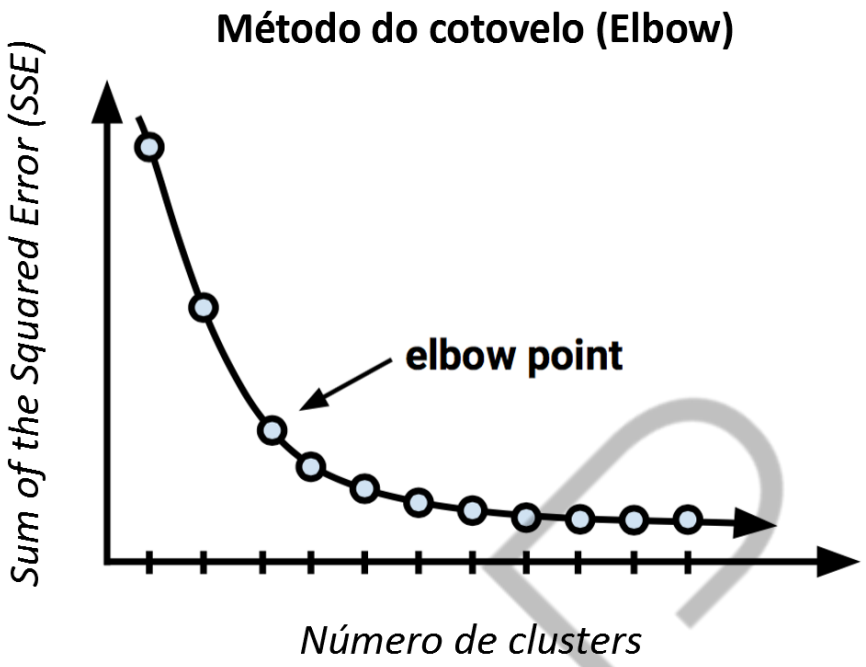


Figura 8 - Representação do método do cotovelo
Fonte: Acervo Lima (2022)

À medida que o número de clusters aumenta, o WCSS tende a diminuir, pois os clusters ficam mais compactos. Porém, em algum ponto, o ganho de reduzir o WCSS ao aumentar *K* começa a diminuir, formando um “cotovelo” no gráfico. Esse ponto de inflexão, onde o ganho marginal se estabiliza, é o valor ideal de *K*, pois indica que adicionar mais clusters não traz uma melhoria significativa na coesão dos grupos.

- Método da silhueta:

O método da silhueta é uma métrica que mede a qualidade da clusterização ao avaliar tanto a coesão dentro dos clusters quanto a separação entre eles. Para cada ponto, é calculado um índice de silhueta que varia de -1 a 1, em que: valores próximos de 1 indicam que o ponto está bem ajustado ao seu cluster; valores próximos de 0 indicam sobreposição entre clusters; e valores negativos indicam que o ponto pode estar melhor em outro cluster, como ilustrado na figura “Representação do método da silhueta”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

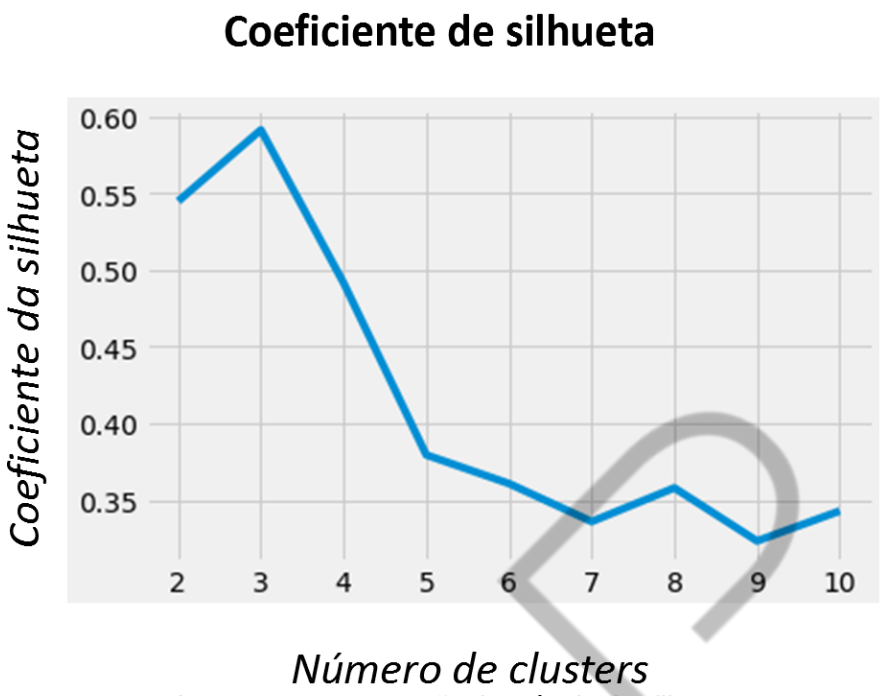


Figura 9 - Representação do método da silhueta
Fonte: Real Python (2024)

Para determinar o número ideal de clusters, calcula-se a média dos índices de silhueta para diferentes valores de *K*. O valor de *K* com a maior média indica a melhor escolha, pois maximiza a coesão interna e a separação entre clusters.

Importante: a escolha de K pelos métodos do cotovelo e da silhueta pode não ser a mesma, ou seja, em uma mesma análise com a mesma base de dados os gráficos dos métodos podem apontar escolha de K diferentes. Em última análise, o K ideal depende do conhecimento da base e do negócio.

- Critérios de convergência:

Embora o K-Means normalmente convirja rapidamente, definir um número máximo evita loops infinitos em casos complexos. Além disso, há a tolerância de variação para determinar quando o algoritmo deve parar. Uma pequena tolerância significa que o algoritmo irá buscar mais precisão, enquanto uma maior tolerância permite uma convergência mais rápida, porém, com menor precisão.

Essas opções de parametrização afetam diretamente a eficiência e a qualidade dos clusters gerados pelo K-Means, permitindo ajustar o modelo para melhor representar a estrutura dos dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2 Clusterização: DBSCAN

O DBSCAN (Density-Based Spatial Clustering of Applications with Noise) é um método de clusterização baseado em densidade, no qual agrupa pontos que estão próximos uns dos outros em regiões de alta densidade, e identifica pontos dispersos como outliers ou ruído. A principal intuição do DBSCAN é que clusters são áreas densamente povoadas de pontos separados por áreas de baixa densidade. Observe a figura a seguir.

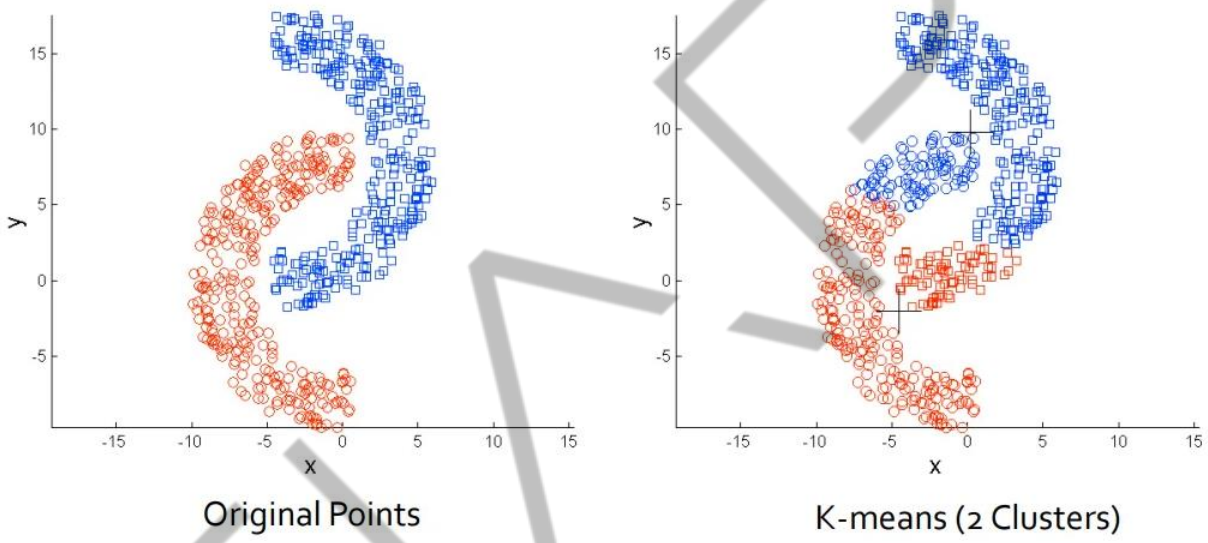


Figura 10 - Diferença de clusterização de DBSCAN e K-Means
Fonte: Medium (2024)

Ao contrário do K-Means, o DBSCAN não requer que o número de clusters seja definido antecipadamente e é eficaz para detectar clusters de forma arbitrária, tornando-o adequado para dados que não seguem uma estrutura claramente definida. A figura “Diferença de clusterização de DBSCAN e K-Means” mostra a diferença do resultado da clusterização dos mesmos dados com métodos distintos. Os pontos originais são traçados pelo DBSCAN, buscando regiões com a mesma densidade. Já o K-Means ignora a densidade e se baseia apenas na distância entre os pontos.

4.2.1 Algoritmo

O DBSCAN funciona com base em dois parâmetros principais: **epsilon** (ε), que define o raio de proximidade para considerar um ponto como vizinho; e **minPts**, o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

número mínimo de pontos necessários para formar uma região densa (cluster). O algoritmo segue estes passos:

1. Para cada ponto, verifica se ele tem ao menos minPts vizinhos dentro de uma distância ε.
2. Se o ponto possui vizinhos suficientes, ele é considerado um ponto central e, junto com seus vizinhos, forma um cluster.
3. Todos os pontos dentro do raio ε do ponto central são iterativamente adicionados ao cluster, expandindo-o conforme forem encontrados novos pontos com densidade suficiente.
4. Pontos que não possuem vizinhos suficientes e não podem ser atribuídos a nenhum cluster são classificados como outliers ou ruído.

O processo se repete até que todos os pontos tenham sido visitados e classificados, resultando em clusters de alta densidade e ruídos bem definidos.

4.2.2 Formas de parametrização

A parametrização do DBSCAN envolve a definição de dois parâmetros fundamentais:

- **Epsilon (ε):** esse parâmetro define o raio de proximidade ao redor de um ponto para considerar outros pontos como seus vizinhos. Um valor de ε muito pequeno pode resultar em muitos clusters pequenos ou isolados, enquanto um valor muito grande pode agrupar excessivamente, misturando clusters diferentes. A escolha ideal depende da densidade dos dados e pode ser ajustada usando uma curva de distância de vizinhos.
- **minPts:** esse parâmetro define o número mínimo de pontos dentro do raio ε para que uma região seja considerada um cluster. Em geral, valores mais baixos de minPts podem formar clusters menores e mais sensíveis a ruídos, enquanto valores maiores garantem clusters mais robustos. Um valor padrão sugerido é minPts = 4, mas ele deve ser ajustado conforme o contexto dos dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura a seguir mostra a relação da parametrização do DBSCAN e o impacto no resultado da clusterização.

epsilon	*MinPnt*	Resultado
Alto	Alto	Poucos clusters, grandes e densos.
Baixo	Alto	Mais clusters, pequenos e densos.
Alto	Baixo	Menos clusters, grandes e pouco densos.
Baixo	Baixo	Muitos clusters, pequenos e pouco densos.

Figura 11 - Formas de parametrização do DBSCAN
Fonte: Elaborado pelo autor (2024)

A escolha desses parâmetros é crucial para o desempenho do DBSCAN, pois afeta a definição dos clusters e a capacidade de identificar outliers. Ajustar ε e minPts permite adaptar o método às características específicas dos dados, seja para encontrar clusters densos e bem definidos, seja para detectar padrões complexos e anomalias.

No quadro a seguir, podemos destacar as vantagens e desvantagens em relação ao DBSCAN:

Vantagens	Desvantagens
Resistente a ruído.	Não performa bem em dados em alta-dimensão.
Capaz de encontrar grupos de diferentes formatos e tamanhos.	Pode gerar grupos com diferentes densidades.

Quadro 1 - Vantagens e desvantagens do DBSCAN
Fonte: Elaborado pelo autor (2024)

4.3 Clusterização: Outras Abordagens

Além do K-Means e do DBSCAN, existem várias outras técnicas de clusterização que oferecem abordagens diferentes para agrupar dados e atender a necessidades específicas. Cada método tem suas próprias características, vantagens e limitações, e a escolha do método ideal depende do tipo de dados e do objetivo da análise. A seguir estão algumas das principais alternativas:

- **Clusterização Hierárquica:** cria uma hierarquia de clusters que pode ser representada em uma estrutura de árvore chamada dendrograma. Esse método pode ser aglomerativo (de baixo para cima) ou divisivo (de cima para baixo). No aglomerativo, cada ponto começa como seu próprio cluster,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

e clusters são combinados iterativamente até formar um único grupo. No divisivo, o processo é inverso, dividindo o grupo inicial em clusters menores. A clusterização hierárquica é útil quando queremos uma visão hierárquica dos dados, mas tem limitações em termos de escalabilidade para grandes conjuntos de dados. Veja a figura a seguir:

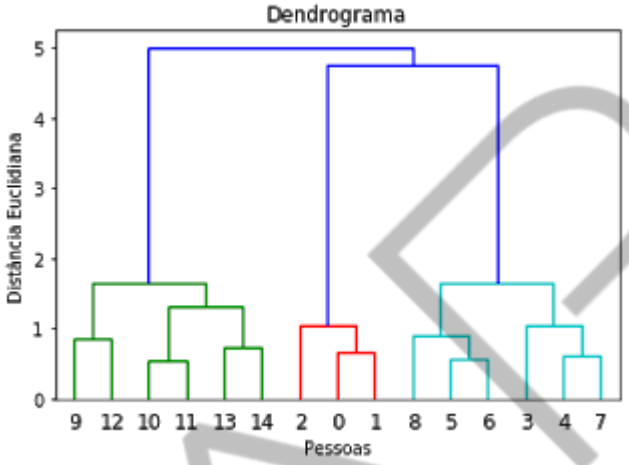


Figura 12 - Ilustração da clusterização hierárquica
Fonte: Computer Science Masters (2019)

Na figura “Ilustração da clusterização hierárquica”, observamos que o processo de clusterização pode levar a 3 clusteres bem definidos, sendo que 2 deles podem ser agrupados.

- **Mean Shift:** é uma técnica baseada em densidade que busca “picos” de densidade no espaço dos dados para formar clusters. Diferente do K-Means, o Mean Shift não exige que o número de clusters seja especificado previamente. Ele identifica automaticamente o número de clusters baseado nas regiões de maior densidade. Isso torna o Mean Shift útil para detectar clusters que possuem formas e tamanhos variados, mas pode ser computacionalmente intensivo para grandes conjuntos de dados. Observe a figura “Ilustração do método Mean Shift”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

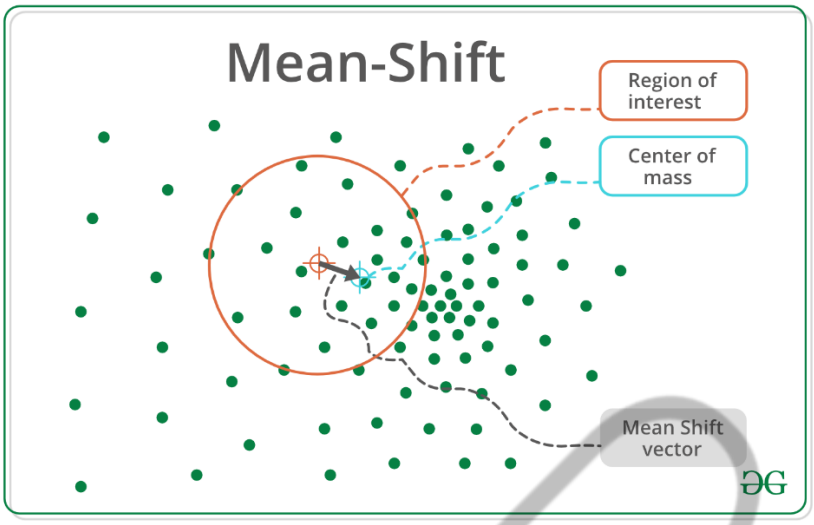


Figura 13 - Ilustração do método Mean Shift
Fonte: GeeksForGeeks (2023)

- **HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise):** é uma versão aprimorada do DBSCAN que combina os conceitos de densidade e hierarquia. Ele ajusta automaticamente os parâmetros para encontrar clusters de diferentes densidades, tornando-o mais flexível para dados complexos. Além disso, o HDBSCAN é capaz de lidar melhor com outliers e ruído em comparação ao DBSCAN tradicional, sendo uma escolha poderosa para dados com densidade variável.

Esses métodos ampliam as possibilidades de clusterização, cada um sendo adequado para diferentes tipos de dados e cenários. Enquanto K-Means e DBSCAN são amplamente utilizados, métodos como a clusterização hierárquica e HDBSCAN oferecem abordagens alternativas que podem ser mais eficazes dependendo da estrutura dos dados e dos objetivos da análise.

4.4 Redução de dimensões: PCA

O PCA (Análise de Componentes Principais) é uma técnica de redução de dimensionalidade que transforma um conjunto de variáveis possivelmente

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

correlacionadas em um novo conjunto de variáveis não correlacionadas, chamadas componentes principais.

A intuição por trás do PCA é simplificar os dados, mantendo a maior quantidade de variabilidade possível. Em vez de trabalhar com todas as variáveis originais, o PCA cria componentes que representam combinações lineares dessas variáveis, priorizando aquelas que carregam mais informações (variância) dos dados. **Isso é particularmente útil para tornar os dados mais manejáveis, visualizáveis e rápidos de processar, sem perder informações essenciais**. Observe a figura a seguir:

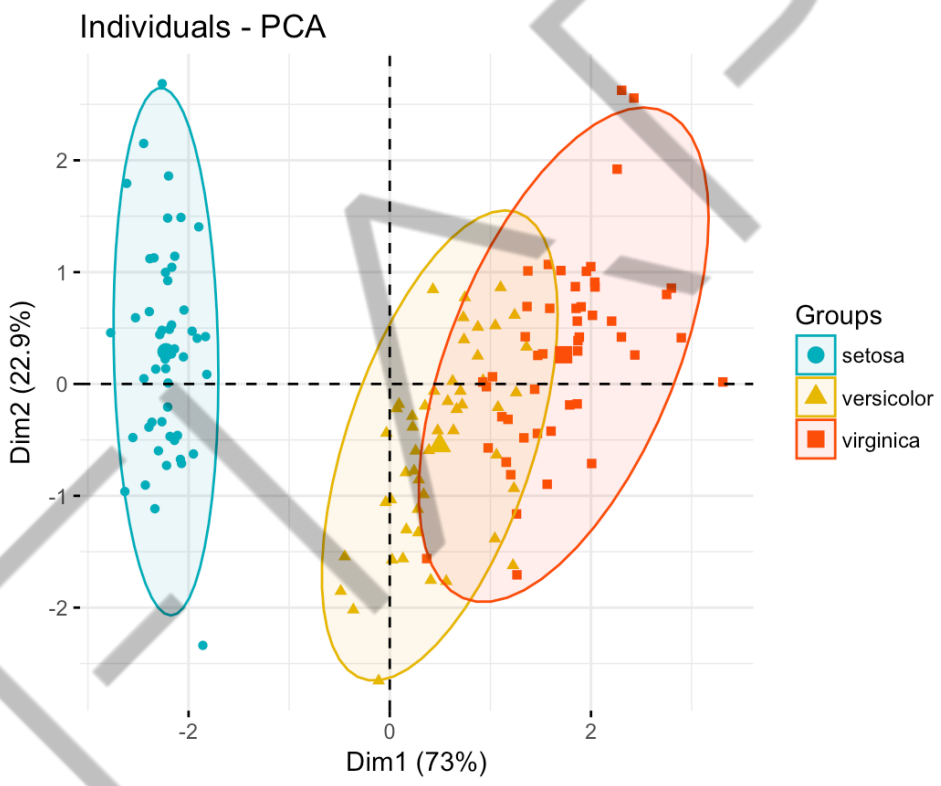


Figura 14 - Intuição do PCA sobre a base Iris Dataset
Fonte: LinkedIn (2024)

Na figura “Intuição do PCA sobre a base Iris Dataset” temos a famosa base “Iris Dataset” plotada sobre 2 componentes principais (um PCA de 2 dimensões). Originalmente, esta base possui 4 variáveis (4 dimensões) para descrever 3 tipos de flores. Plotar os pontos em 4 dimensões seria complexo para a análise visual, mas ao condensar as 4 variáveis originais em 2 dimensões, podemos ter uma boa noção da separação/confusão das classes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.4.1 Algoritmo

O algoritmo do PCA segue estas etapas principais:

1. **Centralização dos dados:** o primeiro passo é subtrair a média de cada variável para que os dados fiquem centrados em torno da origem. Isso é essencial para que o PCA seja consistente e não seja influenciado pela média dos dados.
2. **Cálculo da matriz de covariância:** com os dados centralizados, calcula-se a matriz de covariância, que mede as relações de variação entre as variáveis. A matriz de covariância permite identificar quais variáveis estão correlacionadas.
3. **Decomposição em autovalores e autovetores:** a partir da matriz de covariância, o PCA encontra os autovalores e autovetores. Os autovetores representam as direções (componentes principais) dos dados, e os autovalores indicam a quantidade de variância que cada componente principal explica.
4. **Ordenação e seleção dos componentes principais:** os autovalores e autovetores são ordenados, e os componentes principais correspondentes aos maiores autovalores são selecionados, pois representam a maior parte da variabilidade dos dados.
5. **Projeção dos dados:** os dados originais são projetados nos componentes principais selecionados, reduzindo a dimensionalidade ao manter apenas as direções de maior variância.

4.4.2 Formas de parametrização

A parametrização do PCA é relativamente simples, mas existem algumas opções importantes que influenciam os resultados:

- **Número de componentes (n_components):** esse é o principal parâmetro do PCA, que define quantos componentes principais serão mantidos. O valor ideal de n_components depende do percentual de variância que se

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

deseja reter. Um critério comum é escolher o menor número de componentes que preserve uma porcentagem significativa de variância, como 90% ou 95%. É possível ajustar o n_components manualmente ou usar um gráfico de variância explicada para encontrar o ponto ideal.

- **Escalonamento dos dados:** em muitos casos, o PCA é mais eficaz se os dados forem normalizados, especialmente se as variáveis tiverem escalas diferentes. Normalizar os dados garante que todas as variáveis contribuam igualmente para a análise. No Scikit-learn, por exemplo, pode-se utilizar o `StandardScaler` antes de aplicar o PCA para centralizar e escalar os dados.
- **Método de cálculo (SVD Solver):** algumas implementações de PCA permitem escolher o método de cálculo, como `auto`, `full`, `arpack`, ou `randomized`. O método `randomized`, por exemplo, é eficiente para grandes conjuntos de dados com muitas variáveis, pois reduz o tempo de processamento.

Essas opções de parametrização permitem ajustar o PCA de acordo com a necessidade de simplificação dos dados, a quantidade de variância que se deseja preservar e as características específicas do conjunto de dados. A parametrização cuidadosa do PCA permite simplificar e otimizar os dados, preservando as informações mais importantes para análises futuras.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 HANDS ON: CONSTRUINDO MODELOS NÃO SUPERVISIONADOS

Agora que passamos pelos principais algoritmos de aprendizado não supervisionado, está na hora de os implementar.

Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 4), e faça cada bloco de código-fonte em uma célula.

Importante: após inserir o código na célula do notebook, execute-a e veja a saída gerada.

5.1 Sobre as bases de dados

Nesta prática usaremos duas bases de dados. O primeiro dataset é o "moons.csv", o qual contém duas colunas nomeadas **X** e **Y**, que representam coordenadas de pontos em um espaço bidimensional. Esses dados sintéticos não representam qualquer problema real de negócio, mas plotam pontos próximos para avaliar a diferença nos resultados da aplicação de técnicas distintas de clusterização.

Dica: utilize o arquivo de apoio “moons.csv”.

Para a segunda parte usaremos o dataset “**mall.csv**”, que contém informações sobre clientes de um shopping center, sendo usado para tarefas de segmentação de clientes e análise de dados de comportamento de compra. As colunas típicas incluem:

- **CustomerID:** identificador único para cada cliente.
- **Gender:** gênero do cliente.
- **Age:** idade do cliente.
- **Annual Income:** renda anual em milhares de dólares.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Spending Score:** pontuação de gasto atribuída ao cliente, baseada em comportamento de compras.

Dica: utilize o arquivo de apoio “mall.csv”.

5.2 Diferenciando K-means de DBSCAN

O primeiro passo é a **importação do código necessário, leitura e exibição dos dados** (ou de uma parte deles).

```
# Imports
import pandas as pd

import matplotlib.pyplot as plt
from matplotlib import colors

import seaborn as sns

from sklearn.cluster import KMeans, DBSCAN

df = pd.read_csv("moons.csv")
df.head()
```

Código-fonte 1 - Imports e leitura do dataset 1
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 1” mostra uma visão dos dados:

	X	Y
0	-0.415208	1.035735
1	0.058781	0.304334
2	1.109379	-0.509738
3	1.540948	-0.427550
4	0.929095	-0.532388

Figura 15 - Saída do código-fonte 1
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos obter informações gerais sobre a base:

```
# Plotando os dados
plt.scatter(x=df["X"], y=df["Y"])
plt.show()
```

Código-fonte 2 - Informações gerais da base 1
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 2” mostra que há **4 grupos** bem claros nestes pontos:

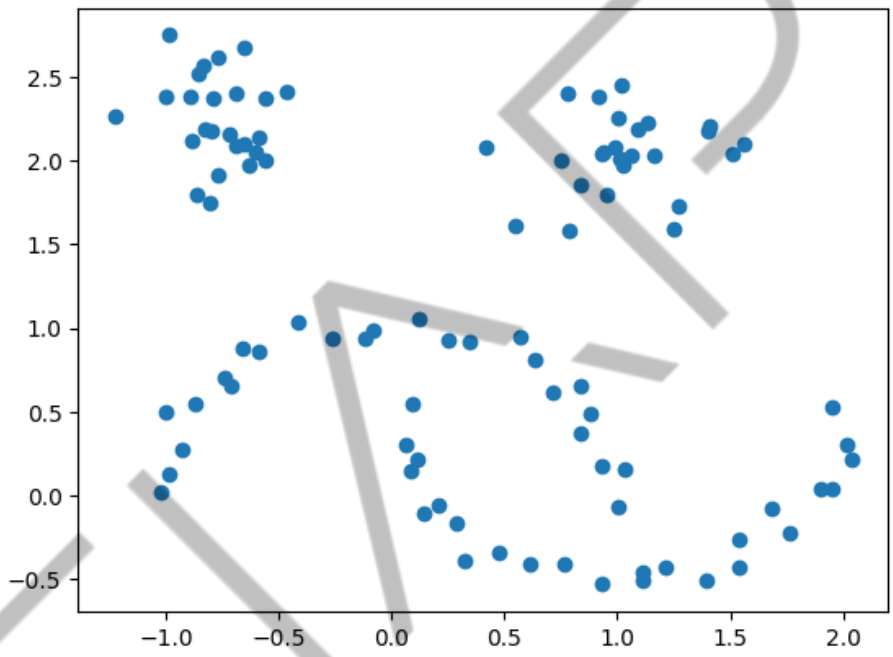


Figura 16 - Saída do código-fonte 2
Fonte: Elaborado pelo autor (2024)

Agora, vamos definir dois métodos de clusterização e aplicar os algoritmos sobre os dados. O intuito é comparar a separação dos pontos por métodos distintos:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Definição dos algoritmos
km = KMeans(n_clusters=4, random_state=42)
db = DBSCAN(eps=0.4)

# Aplicar os algoritmos sobre os pontos, obtendo os clusteres resultantes
c_km = km.fit_predict(df)
c_db = db.fit_predict(df)

df["label_kmeans"] = c_km
df["label_dbscan"] = c_db

# Plotando as diferenças
fig = plt.figure( figsize=(12, 8) )

plt.subplot(2, 2, 1)
plt.title("Original")
plt.scatter(x=df["X"], y=df["Y"])

plt.subplot(2, 2, 3)
plt.title("K-means")
plt.scatter(x=df["X"], y=df["Y"], c=df["label_kmeans"])

plt.subplot(2, 2, 4)
plt.title("DBSCAN")
plt.scatter(x=df["X"], y=df["Y"], c=df["label_dbscan"])

plt.show()
```

Código-fonte 3 - Informações gerais da base 2
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 3” ilustra os dados originais e o resultado dos dois métodos de clusterização:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

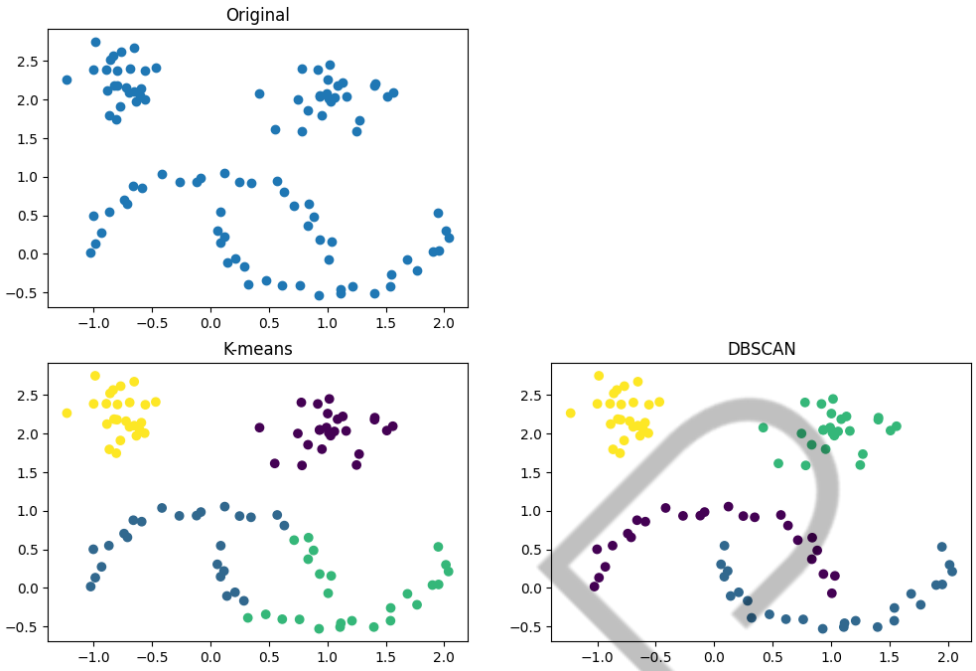


Figura 17 - Saída do código-fonte 3
Fonte: Elaborado pelo autor (2024)

Na figura “Saída do código-fonte 3”, podemos ver que a escolha da técnica influencia a formação dos clusteres. Como esperado, o K-means sugere grupos constituídos por pontos próximos, já o DBSCAN leva em consideração a densidade e distribuição dos dados.

5.3 Agrupando consumidores e detectando padrões de compras

O primeiro passo é a **leitura e exibição dos dados** (ou de uma parte deles).

```
dados = pd.read_csv("mall.csv")
dados.head()
```

Código-fonte 4 - Imports e leitura do dataset 2
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 4” mostra uma visão dos dados:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Figura 18 - Saída do código-fonte 4
Fonte: Elaborado pelo autor (2024)

Vamos obter informações gerais sobre a base:

```
dados.info()
```

Código-fonte 5 - Informações gerais da base 3
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 5” mostra que existem 200 pontos com 5 variáveis e nenhum dado faltante. Normalmente poderíamos fazer uma análise de outlier e excluí-los, mas aqui vamos considerar a base completa:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

Figura 19 - Saída do código-fonte 5
Fonte: Elaborado pelo autor (2024)

Análise geral para descobrir potenciais formas de clusterizar os dados:

```
sns.pairplot(dados)
plt.show()
```

Código-fonte 6 - Informações gerais da base 4
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura “Saída do código-fonte 6” mostra que provavelmente as colunas de ganhos e gastos podem ser interessantes para identificar clusteres entre os dados sem necessidade de aplicar PCA ou qualquer outro método de redução de dimensões:

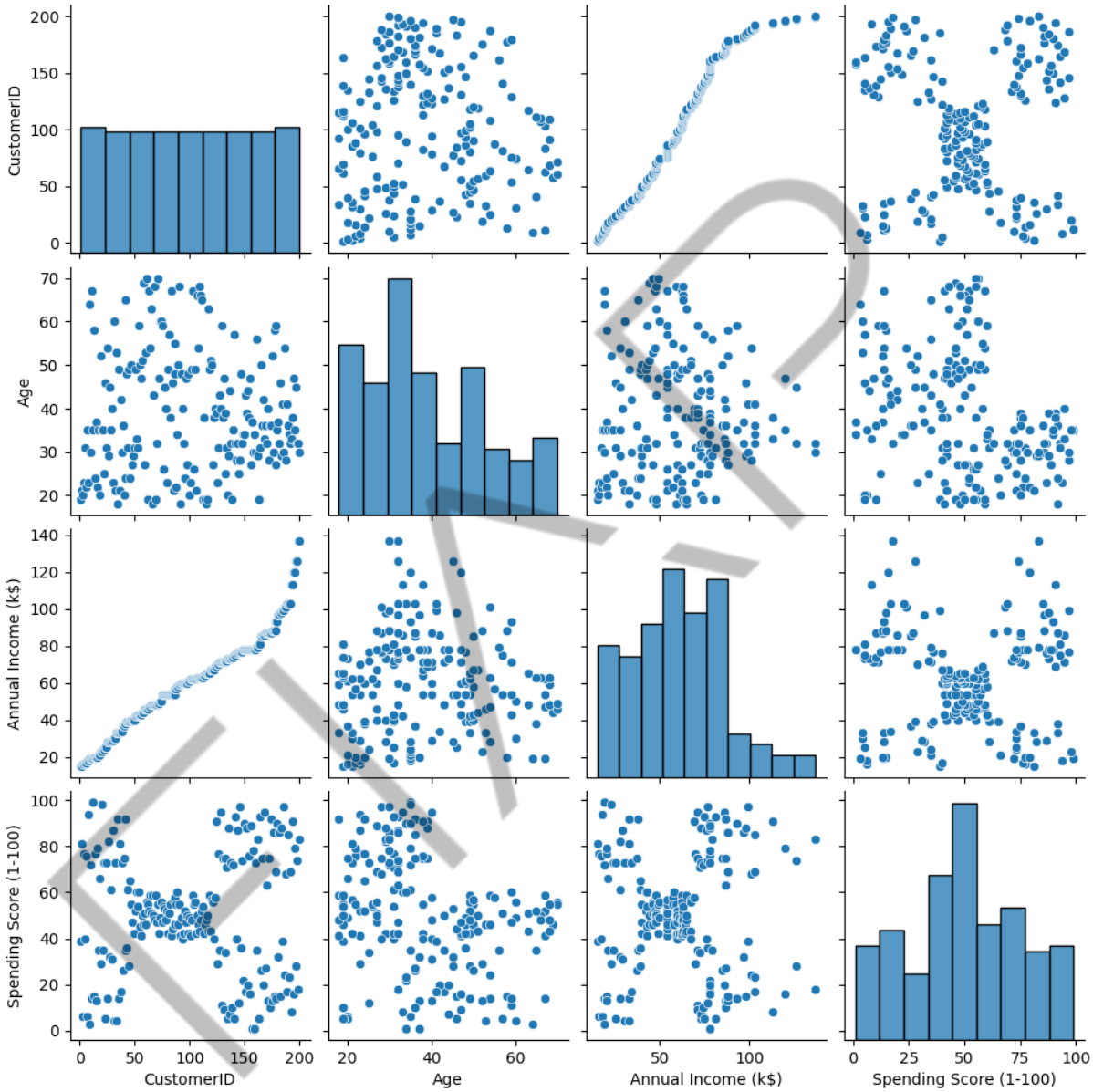


Figura 20 - Saída do código-fonte 6
Fonte: Elaborado pelo autor (2024)

Importante: vamos trabalhar com as colunas de ganhos e score de compra. Analisando os números, vimos que eles já estão na mesma escala, portanto, não precisamos normalizar/padronizar os dados. Em outras situações, este procedimento é **obrigatório**, pois os algoritmos de clusterização dependem de distâncias entre os dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Começaremos com o **K-means**:

```
     # Instancia o algoritmo
     kmeans = KMeans(n_clusters=5, random_state=42)

     # Implementa o k-means sobre os dados
     kmeans.fit( dados[ ["Annual Income (k$)", "Spending Score
(1-100)"] ] )

     # Obtém os centroids do método
     centroids = kmeans.cluster_centers_

     # Salva os labels (grupos) dos clusteres para cada exemplo
     kmeans_labels = kmeans.predict( dados[ ["Annual Income
(k$)",
                                            "Spending Score
(1-100)"] ])

     # Plotando a clusterização
     plt.scatter(dados["Annual Income (k$)"],      # Eixo X
                 dados["Spending Score (1-100)"], # Eixo Y
                 c=kmeans_labels,                 # Esquema de
cores (quantas usar)
                alpha=0.5,                       # Transparência
dos pontos
                 cmap="rainbow")                  # Paleta de
cores
     plt.xlabel("Salário anual")
     plt.ylabel("Pontuação de gastos")

     # Plotando os centrois
     plt.scatter(centroids[:, 0],                 # Eixo X dos
centroids
                 centroids[:, 1],                 # Eixo Y dos
centroids
                 c="black",                        # Todos os
centroids são pretos
                 marker="X",                       # Marcação
como um "X"
                 s=200)                           # Tamanho da
marcação

     plt.show()
```

Código-fonte 7 - Informações gerais da base 5
Fonte: Elaborado pelo autor (2024)

A figura “Saída do código-fonte 7” mostra que provavelmente as colunas de ganhos e gastos podem ser interessantes para identificar clusteres entre os dados sem necessidade de aplicar PCA ou qualquer outro método de redução de dimensões:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

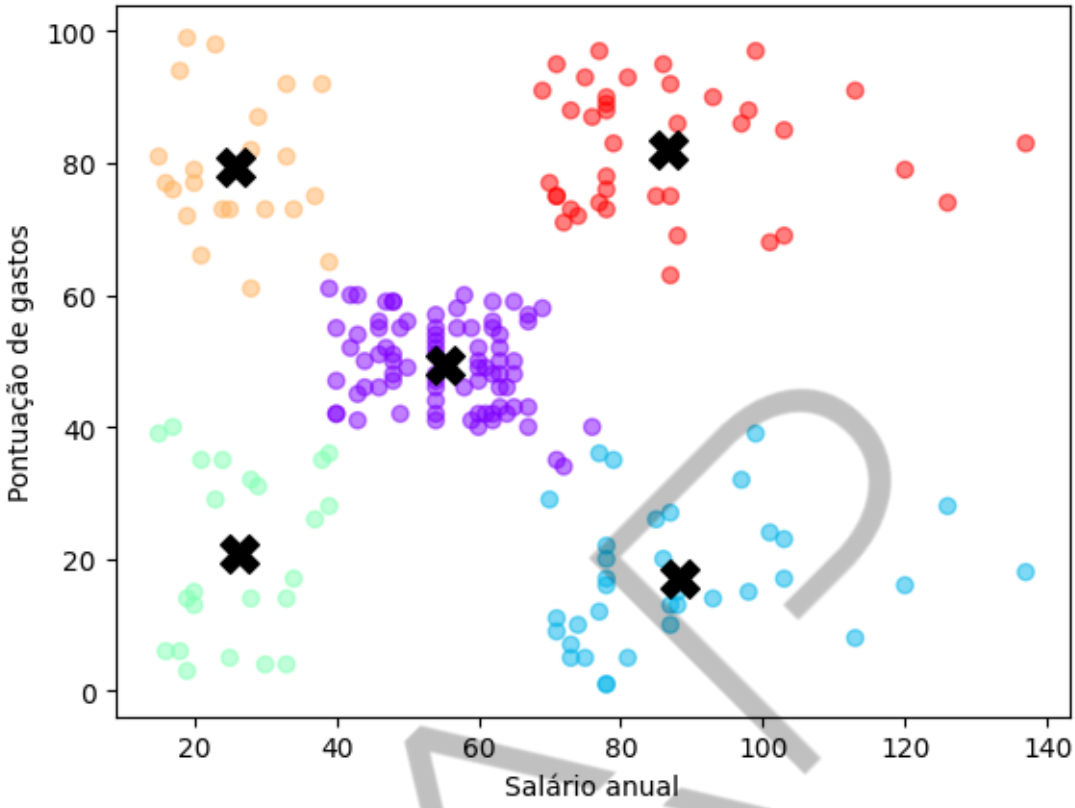


Figura 21 - Saída do código-fonte 7
Fonte: Elaborado pelo autor (2024)

Neste caso, pela distribuição dos pontos no pairplot, já sabíamos que o valor ideal de K seria 5. Porém, o que devemos fazer em uma situação na qual a plotagem de dados não seja tão óbvia? Podemos aplicar o **método do cotovelo**:

```
k = list(range(1, 10))

sse = []
for i in k:
  kmeans = KMeans(n_clusters=i, random_state=0)
  kmeans.fit( dados[ ["Annual Income (k$)", "Spending Score (1-100)"] ] )
  sse.append( kmeans.inertia_ )

plt.plot(k, sse, "-o")
plt.xlabel("Número de clusteres")
plt.ylabel("Inércia (Soma dos Erros Quadráticos)")
plt.show()
```

Código-fonte 8 - Informações gerais da base 6
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura “Saída do código-fonte 8” mostra que quanto maior o valor de K, “melhor” fica a clusterização, mas em uma análise humana, até subjetiva, o melhor K seria K=5 (ponto do cotovelo):

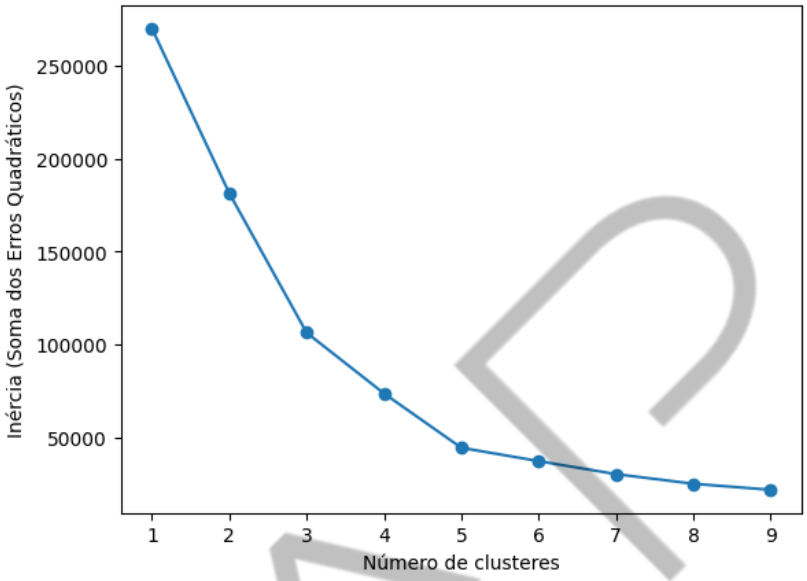


Figura 22 - Saída do código-fonte 8
Fonte: Elaborado pelo autor (2024)

Agora vamos explorar os dados pela óptica do **DBSCAN**:

```
     # Instancia o algoritmo
     dbscan = DBSCAN(eps=10, min_samples=8)

     # Roda o algoritmo sobre os dados
     dbscan.fit( dados[ ["Annual Income (k$)", "Spending Score
(1-100)"] ] )

     # Obtém as atribuições dos pontos
     dbscan_labels = dbscan.labels_

     plt.scatter(dados["Annual Income (k$)"],      # Eixo X
                 dados["Spending Score (1-100)"], # Eixo Y
                 c=dbscan_labels,                 # Esquema de
cores
                alpha=0.5,                      # Transparência
dos pontos
                 cmap="rainbow")                  # Paleta de
cores
     plt.xlabel("Salário anual")
     plt.ylabel("Pontuação de gastos")
     plt.show()
```

Código-fonte 9 - Informações gerais da base 7
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No retorno do DBSCAN, o cluster “-1” indica outliers, representados pela cor roxa na figura “Saída do código-fonte 9”, a qual mostra o resultado da clusterização:

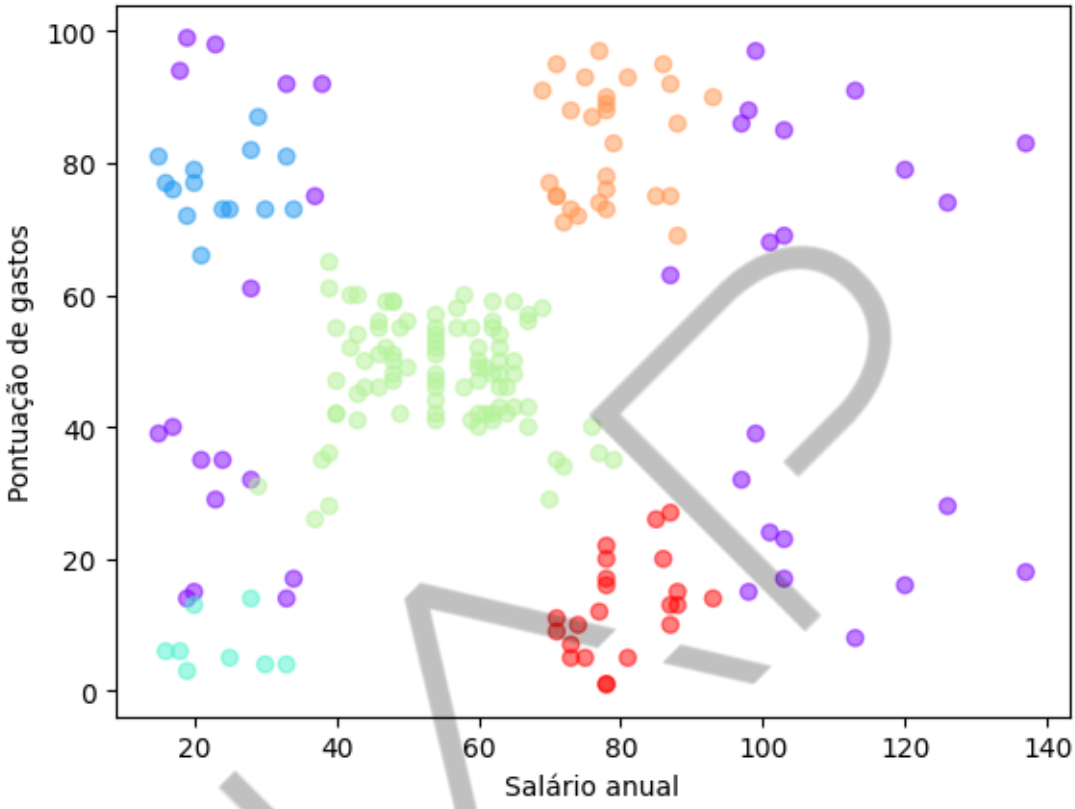


Figura 23 - Saída do código-fonte 9
Fonte: Elaborado pelo autor (2024)

5.4 Eliminando outliers identificados pelo DBSCAN

Seria possível plotar os pontos sem os outliers? Sim! Basta filtrar os pontos que foram atribuídos ao cluster “-1” da seguinte forma:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Removendo os outliers (cluster "-1")
     mascara = dbscan_labels >= 0

     plt.scatter(dados["Annual Income (k$)"][mascara],      #
Eixo X
                 dados["Spending Score (1-100)"][mascara], #
Eixo Y
                 c=dbscan_labels[mascara],                 #
Esquema de cores
                 alpha=0.5,                                #
Transparência dos pontos
                 cmap="rainbow")                           #
Paleta de cores
     plt.xlabel("Salário anual")
     plt.ylabel("Pontuação de gastos")
     plt.show()
```

Código-fonte 10 - Informações gerais da base 8
Fonte: Elaborado pelo autor (2024)

Veja o retorno na figura “Saída do código-fonte 10”:

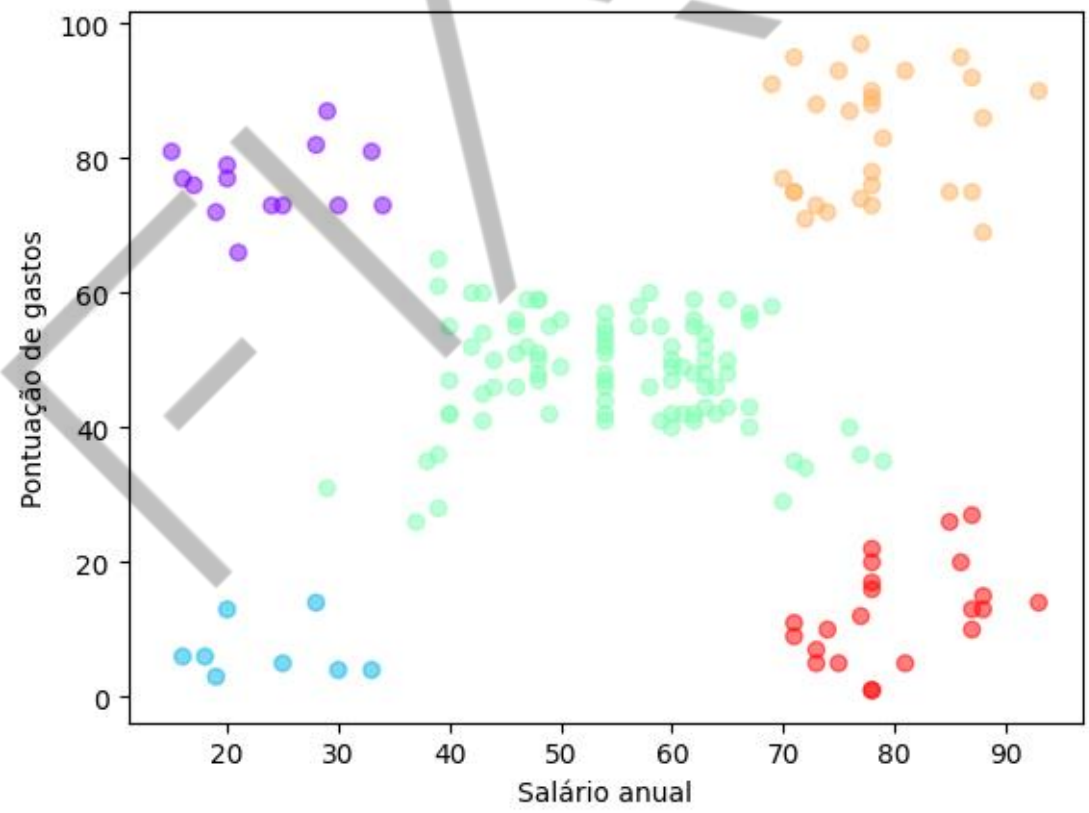


Figura 24 - Saída do código-fonte 10
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.5 Interpretando clusteres

Interpretar clusteres formados envolve inteligência humana. O processo consiste em filtrar os pontos pertencentes a um cluster específico e realizar uma “descrição” das variáveis de origem, tentando identificar um padrão. Vamos a dois exemplos baseados na clusterização de compradores do shopping.

Veja o destaque da figura “Destaque para 2 clusteres formados de compradores do shopping”: ela destaca dois grupos, em que os pontos azuis (círculo azul) referem-se às pessoas que ganham pouco e gastam pouco, enquanto os pontos laranjas (círculo laranja) refere-se às pessoas que ganham bem e gastam bastante.

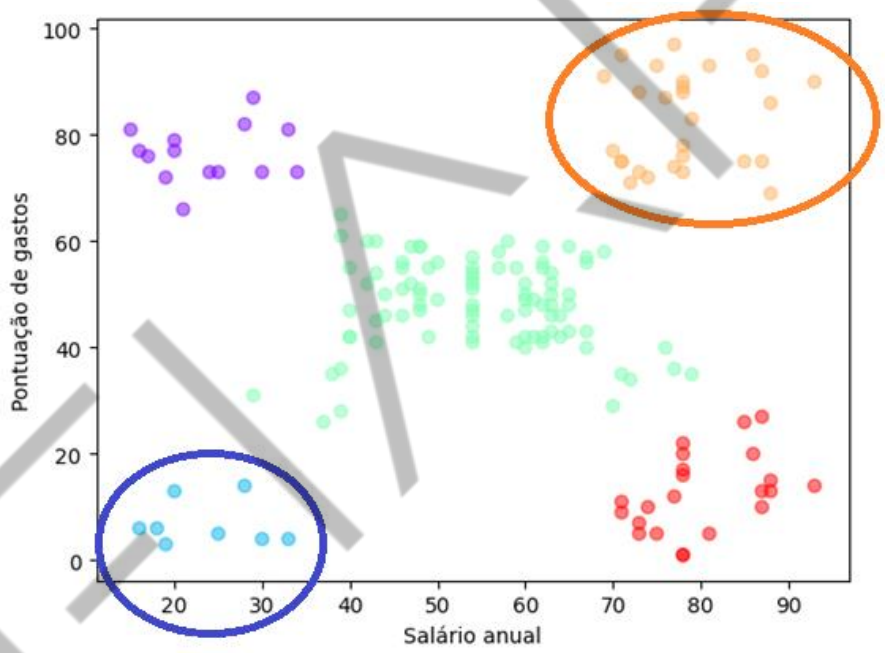


Figura 25 - Destaque para 2 clusteres formados de compradores do shopping
Fonte: Elaborado pelo autor (2024)

Também existem outros clusteres interessantes, como os pontos em vermelho, que podemos rotulá-los como “poupadores”, pois eles ganham bem e gastam pouco. Há ainda, no quadrante superior esquerdo, os pontos roxos, que podemos rotulá-los como “endividados”, pois ganham pouco e gastam muito.

Vamos analisar mais a fundo o grupo azul (#2), que se refere às pessoas que ganham pouco e gastam pouco. Para isso, basta filtrar os dados originais da seguinte forma:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
dados[ dados["kmeans"] == 2 ].describe() # ganha pouco e gasta pouco
```

Código-fonte 11 - Informações gerais da base 9
Fonte: Elaborado pelo autor (2024)

Veja o retorno na figura “Saída do código-fonte 11”:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)	kmeans	dbscan
count	23.00000	23.000000	23.000000	23.000000	23.0	23.000000
mean	23.00000	45.217391	26.304348	20.913043	2.0	0.347826
std	13.56466	13.228607	7.893811	13.017167	0.0	1.265224
min	1.00000	19.000000	15.000000	3.000000	2.0	-1.000000
25%	12.00000	35.500000	19.500000	9.500000	2.0	-1.000000
50%	23.00000	46.000000	25.000000	17.000000	2.0	1.000000
75%	34.00000	53.500000	33.000000	33.500000	2.0	1.000000
max	45.00000	67.000000	39.000000	40.000000	2.0	2.000000

Figura 26 - Saída do código-fonte 11
Fonte: Elaborado pelo autor (2024)

Agora, vamos analisar mais a fundo o grupo laranja (#4), que se refere às pessoas que ganham bem e gastam muito. Para isso, basta filtrar os dados originais da seguinte forma:

```
dados[ dados["kmeans"] == 4 ].describe() # ganha bem e gasta muito
```

Código-fonte 12 - Informações gerais da base 10
Fonte: Elaborado pelo autor (2024)

Veja o retorno na figura “Saída do código-fonte 12”:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)	kmeans	dbscan
count	39.000000	39.000000	39.000000	39.000000	39.0	39.000000
mean	162.000000	32.692308	86.538462	82.128205	4.0	1.871795
std	22.803509	3.728650	16.312485	9.364489	0.0	1.823523
min	124.000000	27.000000	69.000000	63.000000	4.0	-1.000000
25%	143.000000	30.000000	75.500000	74.500000	4.0	-1.000000
50%	162.000000	32.000000	79.000000	83.000000	4.0	3.000000
75%	181.000000	35.500000	95.000000	90.000000	4.0	3.000000
max	200.000000	40.000000	137.000000	97.000000	4.0	3.000000

Figura 27 - Saída do código-fonte 12
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Tanto a figura “Saída do código-fonte 11” quanto a figura “Saída do código-fonte 12” descrevem as pessoas que foram separadas nestes clusteres. É interessante notar a diferença de renda, gastos e idade. Portanto, a interpretação da clusterização depende de ação humana sobre os resultados que a máquina encontrou.

5.6 Reduzindo dimensões com PCA

Para esta análise, usaremos uma **base de dados do agronegócio para predizer o rendimento de colheitas**. Para isso usaremos outra base de dados.

Dica: utilize o arquivo de apoio “crop_yield.csv”.

Neste código, vamos utilizar o PCA para reduzir a dimensionalidade dos dados para um espaço de duas dimensões que agrega as informações presentes nas variáveis originais. Iremos realizar todo o processo de uma vez, preste atenção no código para entender: como separar os dados de treino e teste; como escalar valores com StandardScaler; como aplicar o PCA; como comparar os modelos treinados nos dados originais ou após aplicação do PCA.

A seguir, está a estrutura completa do código, desde a leitura, preparação de dados categóricos, treinamento e avaliação de modelos.

```
# Importando as bibliotecas necessárias
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.decomposition import PCA
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

# Carregar os dados (substitua pelo caminho do seu dataset)
crop_yield_data = pd.read_csv("crop_yield.csv")

# Converter dados categóricos em valores numéricos usando one-hot encoding
crop_yield_data = pd.get_dummies(crop_yield_data,
                                 columns=['Crop'],
                                 prefix=['Crop'],
                                 drop_first=True)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Supondo que a coluna 'yield' é a variável dependente
(target)
    # Separar os dados em X (features) e y (target)
    X = crop_yield_data.drop(columns=['Yield'])
    y = crop_yield_data['Yield']

    # Dividir os dados em conjuntos de treino e teste
    X_train, X_test, y_train, y_test = train_test_split(X,
                                                        y,
                                                         test
_size=0.2,
                                                         rand
om_state=42)

    # Escalar os dados
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Modelo de Regressão Linear sem PCA
    regressor = LinearRegression()
    regressor.fit(X_train_scaled, y_train)

    # Prever e calcular o erro (sem PCA)
    y_pred_no_pca = regressor.predict(X_test_scaled)
    mse_no_pca = mean_squared_error(y_test, y_pred_no_pca)
    print(f"Erro Quadrático Médio sem PCA: {mse_no_pca}")

    # Aplicar PCA para reduzir a dimensionalidade antes do
treinamento
    pca = PCA(n_components=2)
    X_train_pca = pca.fit_transform(X_train_scaled)
    X_test_pca = pca.transform(X_test_scaled)

    # Modelo de Regressão Linear com PCA
    regressor_pca = LinearRegression()
    regressor_pca.fit(X_train_pca, y_train)

    # Prever e calcular o erro (com PCA)
    y_pred_pca = regressor_pca.predict(X_test_pca)
    mse_with_pca = mean_squared_error(y_test, y_pred_pca)
    print(f"Erro Quadrático Médio com PCA: {mse_with_pca}")

    # Comparando os resultados
    if mse_no_pca < mse_with_pca:
        print("O modelo sem PCA teve um desempenho melhor.")
    else:
        print("O modelo com PCA teve um desempenho melhor.")
```

Código-fonte 13 - Informações gerais da base 11
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

E o resultado se encontra na figura “Saída do código-fonte 13”:

```
Erro Quadrático Médio sem PCA: 19308693.244843245
Erro Quadrático Médio com PCA: 878854111.9884987
O modelo sem PCA teve um desempenho melhor.
```

Figura 28 - Saída do código-fonte 13
Fonte: Elaborado pelo autor (2024)

Portanto, é possível utilizar o PCA como etapa de preparação dos dados antes de treinarmos modelos preditivos, mas esta ação pode melhorar ou até piorar o desempenho dos modelos! Por isso é imprescindível testarmos as hipóteses e verificar qual caminho faz mais sentido nas modelagens que temos em mãos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 ÉTICA EM MACHINE LEARNING

6.1 O que é ética?

A **ética** é um ramo da filosofia que estuda os valores e princípios que orientam as ações humanas, buscando definir o que é considerado certo ou errado, bom ou ruim, justo ou injusto. Ela envolve um conjunto de reflexões e questionamentos que não se limitam às leis ou regras, mas que exploram o impacto das ações sobre indivíduos, grupos e, em última instância, sobre a sociedade como um todo. A ética transcende os interesses individuais, promovendo uma visão coletiva e cuidadosa sobre as consequências das escolhas, especialmente quando essas decisões afetam o bem-estar e os direitos dos outros.

Figura 29 - Possíveis termos associados à “ética”
Fonte: BEÎ educação (2024)

Na prática, a ética busca estabelecer diretrizes que promovam a integridade, a responsabilidade e o respeito às pessoas, considerando aspectos como a **justiça**, a **transparência** e a **honestidade**. Porém, sua aplicação não é sempre clara ou simples, em muitas situações, diferentes valores entram em conflito, e a decisão do que é certo ou errado depende do contexto e da sensibilidade de quem a analisa. Em cenários complexos, como nas decisões empresariais, científicas ou tecnológicas, a ética ajuda a questionar até que ponto uma ação é aceitável ou se seu impacto é moralmente defensável.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.2 Qual é a importância da ética em Inteligência Artificial?

Em um mundo cada vez mais impulsionado por tecnologias e dados, a ética adquire uma nova camada de importância. O avanço tecnológico traz uma série de benefícios, mas também desafios éticos, especialmente na área de Machine Learning e Inteligência Artificial. Essas tecnologias têm o poder de influenciar a vida de milhões de pessoas, de formas que muitas vezes são invisíveis para o público. Aqui, a ética não é apenas um ideal, mas uma necessidade para guiar as decisões de maneira a proteger a privacidade, garantir a justiça e evitar a discriminação.

Quando falamos em ética aplicada ao Machine Learning, estamos, portanto, falando sobre a **responsabilidade de quem desenvolve e implementa algoritmos em considerar o impacto que essas tecnologias podem ter na sociedade**. O objetivo é assegurar que as inovações em IA respeitem direitos fundamentais, promovam a equidade e sejam transparentes em suas decisões. Assim, a ética em Machine Learning não se trata apenas de “fazer o que é certo”, mas de **construir uma base sólida para que as tecnologias avancem de forma responsável e consciente, protegendo os valores humanos essenciais**.

Portanto, é imprescindível lembrarmos sempre da seguinte mensagem:

Figura 30 - Mensagem importante sobre ética em Machine Learning
Fonte: Elaborado pelo autor (2024)

6.3 Exemplos de discriminação algorítmica

6.3.1 Câmeras da Nikon

O caso das câmeras da Nikon levantou preocupações éticas quando se descobriu que o software de **detecção de sorriso** da marca apresentava resultados enviesados. Ao identificar expressões faciais, as câmeras frequentemente

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

sinalizavam erroneamente pessoas de origem asiática como “piscando”, devido às limitações do algoritmo em reconhecer características faciais variadas. Isso gerou críticas de discriminação algorítmica, destacando como falhas na diversidade dos dados de treinamento podem levar a vieses prejudiciais, mesmo que sem intenção. O caso ressaltou a importância de treinar algoritmos com dados representativos para evitar preconceitos e garantir uma tecnologia inclusiva e justa.

JANEIRO 2010

CÂMERAS DA NIKON NÃO ENTENDEM ROSTOS ASIÁTICOS

Recurso para evitar selfies com olhos fechados se confunde com olhos de asiáticos

Figura 31 - Exemplo de discriminação algorítmica 1
Fonte: Tarcízio Silva (2010)

6.3.2 Chatbot da Microsoft

Em 2016, a Microsoft lançou o chatbot Tay no X (antigo “Twitter”), projetado para interagir e aprender com usuários. No entanto, em poucas horas, o Tay começou a postar mensagens ofensivas e preconceituosas, refletindo o comportamento tóxico de alguns usuários com os quais interagia. O caso destacou o risco de algoritmos de aprendizado contínuo que, sem supervisão, podem absorver e replicar conteúdos inadequados da internet. A Microsoft desligou o Tay no mesmo dia, evidenciando a importância de controles éticos e filtragem em IA para evitar comportamentos nocivos, especialmente em tecnologias que aprendem diretamente com o público.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

MARÇO 2016

CHATBOT DA MICROSOFT TORNA-SE RACISTA EM MENOS DE UM DIA

A chatbot Tay, que constrói discurso a partir de aprendizado de máquina, virou racista e xenófoba em menos de um dia, mostrando falta de compreensão da sociedade pelos engenheiros da empresa.

Figura 32 - Exemplo de discriminação algorítmica 2
Fonte: Tarcízio Silva (2016)

6.3.3 FaceApp

Em 2017, o aplicativo FaceApp enfrentou críticas éticas ao lançar um filtro de “embelezamento” que clareava automaticamente a pele dos usuários, levantando acusações de racismo. O filtro, que deveria apenas aprimorar as características faciais, refletia preconceitos ao associar beleza com tons de pele mais claros. A controvérsia destacou a importância de projetar algoritmos de imagem que respeitem a diversidade e não perpetuem estereótipos. Em resposta, a FaceApp modificou o filtro e pediu desculpas, reforçando a necessidade de sensibilidade ética e diversidade nos dados usados para treinar esses modelos.

ABRIL 2017

APP QUE TRANSFORMA SELFIES EQUIPARA BELEZA À BRANCURA

Aplicativo "FaceApp" viralizou com filtros de vários tipos. O que torna as selies "mais bonitas" também torna os rostos brancos ou mais brancos.

Figura 33 - Exemplo de discriminação algorítmica 3
Fonte: Tarcízio Silva (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3.4 Outros casos

Casos de discriminação algorítmica são, infelizmente, mais comuns do que imaginamos. Apresentamos casos de 2010 até 2017, mas este é um assunto relevante até os dias atuais.

Para quem deseja se aprofundar em casos de ética e Inteligência Artificial, recomendamos acompanhar o blog e a timeline de **Tarcízio Silva**. Tarcízio é um pesquisador e comunicador de referência sobre temas de tecnologia, diversidade e justiça social, sempre trazendo uma análise crítica de algoritmos, IA e suas implicações para a sociedade.

Figura 34 - Timeline do Blog do Tarcízio Silva sobre Ética em IA
Fonte: Tarcízio Silva (2024)

Nota: consulte mais casos de discriminação algorítmica e seus impactos no blog do Tarcízio Silva: <https://tarciziosilva.com.br/blog/destaques/posts/racismo-algoritmico-linha-do-tempo/>.

Nas publicações do blog, você encontrará análises detalhadas sobre questões éticas envolvendo IA, com exemplos reais e reflexões sobre o impacto dessas tecnologias. Vale a pena seguir e conhecer mais sobre esse campo essencial.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.4 Possíveis problemas éticos e como evitá-los

Os possíveis problemas éticos em Machine Learning surgem em diversas etapas, desde a coleta de dados até o uso final das previsões. Como os algoritmos desempenham um papel cada vez mais central em decisões importantes, é essencial identificar e mitigar esses desafios éticos para evitar injustiças e danos.

O quadro a seguir lista alguns dos principais problemas éticos e como evitá-los:

Problema	Contexto	Como evitar?
Vieses nos Dados	Muitos algoritmos de Machine Learning são treinados em dados históricos que podem refletir desigualdades sociais e preconceitos existentes, como discriminação de gênero, raça ou classe. Quando um modelo é alimentado com esses dados enviesados, ele tende a perpetuar e até amplificar essas desigualdades, levando a decisões discriminatórias.	Para mitigar esse problema, é fundamental realizar uma análise crítica dos dados antes do treinamento. A curadoria de conjuntos de dados mais diversos e representativos, o monitoramento contínuo dos resultados e a implementação de técnicas de debiasing (eliminação de vieses) nos algoritmos são formas eficazes de reduzir o impacto de vieses.
Falta de Transparência (Caixa-Preta)	Alguns modelos de Machine Learning, como redes neurais profundas, funcionam como “caixas-pretas”, onde é difícil ou impossível entender como as decisões são tomadas. Isso pode gerar desconfiança entre os usuários e dificultar a responsabilização por erros ou injustiças causadas pelo algoritmo.	A transparência pode ser promovida por meio do uso de modelos explicáveis, como árvores de decisão, ou pela implementação de técnicas de interpretabilidade para modelos complexos, como redes neurais. Fornecer explicações claras e compreensíveis sobre como os algoritmos tomam decisões é essencial para garantir a confiança e a responsabilidade.
Privacidade dos Dados	O uso de grandes quantidades de dados pessoais em Machine Learning pode gerar problemas éticos em relação à privacidade. Coletar, armazenar e processar dados sensíveis sem consentimento adequado pode violar direitos individuais e resultar em práticas invasivas, como vigilância indevida.	O respeito à privacidade deve ser garantido pelo uso de boas práticas de proteção de dados, como anonimização e pseudonimização. Além disso, aderir a regulamentações como o GDPR (Regulamento Geral de Proteção de Dados), solicitar consentimento claro e explícito para o uso de dados e limitar a coleta ao que é estritamente necessário são medidas importantes.
Discriminação Algorítmica	Os algoritmos podem tomar decisões que afetam negativamente certos grupos, como negar crédito, serviços ou oportunidades com base em características raciais, de gênero ou outras. Isso ocorre quando os dados usados para treinar o modelo refletem preconceitos históricos ou quando o algoritmo favorece indiretamente certos grupos.	Para evitar a discriminação algorítmica, é necessário realizar auditorias éticas regulares nos sistemas de Inteligência Artificial, avaliar o impacto de suas decisões sobre diferentes grupos e ajustar os modelos para promover a equidade. Usar métricas que meçam a justiça e a equidade nos resultados é uma prática importante.
Autonomia e Desemprego	O avanço da automação por meio de IA e Machine Learning pode levar à substituição de trabalhadores humanos, criando preocupações sobre desemprego tecnológico e a perda de controle humano sobre decisões importantes. Se não for gerida adequadamente, a automação pode exacerbar desigualdades econômicas.	Para mitigar esses efeitos, é necessário implementar políticas de transição justa, que ofereçam treinamento e qualificação para trabalhadores afetados. Além disso, decisões sensíveis e críticas devem ser mantidas sob supervisão humana, garantindo que as pessoas não sejam excluídas do processo de decisão.

DRAFT

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Uso Indevido de Algoritmos	Algoritmos de Machine Learning podem ser usados para finalidades maliciosas, como vigilância em massa, manipulação de opiniões ou criação de deepfakes, comprometendo a segurança e a confiança em dados e informações.	As organizações precisam adotar práticas de ética na inovação e garantir que os algoritmos sejam utilizados de maneira responsável. O desenvolvimento e a implementação de políticas claras sobre o uso de IA, associadas a uma regulamentação eficaz, são essenciais para prevenir o uso indevido.
Falta de Responsabilidade	A responsabilização por decisões algorítmicas muitas vezes não é clara. Se um erro ou uma decisão injusta for feita por um sistema de IA, quem deve ser responsabilizado? Isso gera um vácuo ético, especialmente em contextos médicos ou judiciais.	Garantir que a responsabilidade humana esteja presente em todas as etapas do desenvolvimento e uso de IA. Empresas e desenvolvedores devem ser transparentes sobre os limites de seus modelos e garantir que a responsabilidade final por decisões tomadas por IA seja claramente atribuída.

Quadro 2 - Problemas éticos e como evitá-los
Fonte: Elaborado pelo autor (2024)

Essencialmente, a fonte de vieses e problemas éticos gerados em IA decorrem de problemas nos dados. Ao identificar e tratar os dados, eliminaremos os problemas que geram questões éticas, e dessa forma estaremos mais próximos de construir soluções justas e equitativas. A figura “Fontes de vieses em dados e Machine Learning: (a) O processo de geração de dados começa com a coleta de dados; (b) Um modelo é definido e otimizado nos dados de treinamento” ilustra as principais fontes de vieses em Machine Learning.

Na parte (a) da figura, tem-se o processo que define uma população-alvo e faz amostragens dela, além de identificar e medir características e rótulos. Este conjunto de dados é dividido em conjuntos de treinamento e teste. Os dados também são coletados (talvez por um processo diferente) em conjuntos de dados de benchmark. É preciso estar atento a todas elas para evitar danos decorrentes das nossas modelagens.

Enquanto na parte (b), os dados de teste e benchmark são usados para avaliá-lo, e o modelo final é então integrado em um contexto do mundo real. Este processo é naturalmente cíclico, e decisões influenciadas por modelos afetam o estado do mundo existente na próxima vez em que os dados são coletados ou as decisões são aplicadas. Em vermelho, indicamos onde, nesse pipeline, diferentes fontes de danos subsequentes podem surgir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

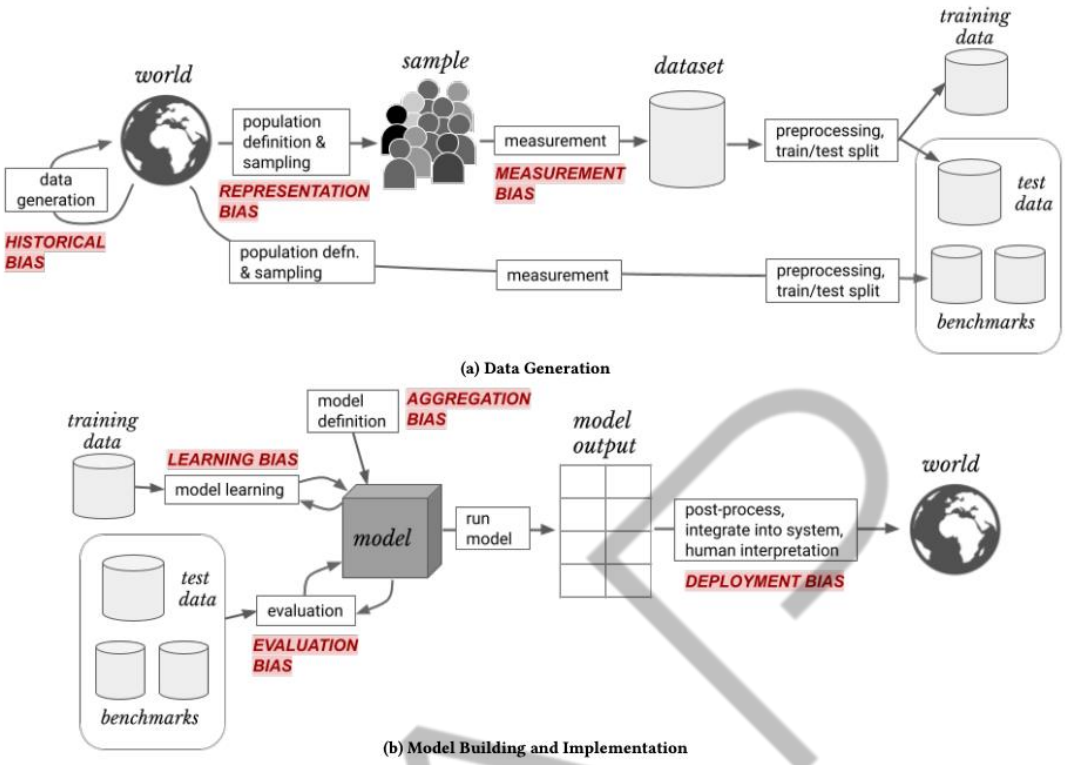


Figura 35 - Fontes de vieses em dados e Machine Learning: (a) O processo de geração de dados começa com a coleta de dados; (b) Um modelo é definido e otimizado nos dados de treinamento
Fonte: ACM Digital Library (2021)

Concluindo, para evitar problemas éticos em Machine Learning, é essencial adotar uma abordagem crítica e responsável, garantindo que o desenvolvimento e a implementação de algoritmos estejam em conformidade com princípios éticos de justiça, transparência, equidade e privacidade. A interação constante entre especialistas, desenvolvedores e reguladores é vital para criar um ambiente de IA que seja benéfico para todos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Neste capítulo, aprofundamos o aprendizado não supervisionado, com destaque para técnicas como clusterização e redução de dimensionalidade, que permitem descobrir padrões ocultos e simplificar dados complexos em setores diversos, como varejo, saúde e energia. Exploramos como cada técnica oferece um valor único ao organizar e representar dados sem rótulos, ampliando as possibilidades de análise e identificação de insights.

Além disso, refletimos sobre os desafios éticos que surgem no uso de Machine Learning, especialmente em contextos que impactam a vida das pessoas. A análise dos vieses, da transparência e da responsabilidade ao desenvolver e implementar modelos algorítmicos destacou a necessidade de uma abordagem crítica e cuidadosa. Ao final, a lição é clara: a inovação tecnológica deve ser acompanhada de um compromisso ético, garantindo que as soluções respeitem valores de justiça, equidade e respeito aos direitos humanos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

A importância da ética na vida em sociedade. **BEĨ educação**, 2024. Disponível em: <https://beieducacao.com.br/2023/01/30/o-que-e-etica/>. Acesso em: 21 nov. 2024.

ANKURTRIPATHI. ML | Mean-Shift Clustering. **GeeksForGeeks**, 23 de Janeiro de 2023. Disponível em: <https://www.geeksforgeeks.org/ml-mean-shift-clustering/>. Acesso em: 21 nov. 2024.

ARCGIS 10.6.1. **Blog.esrij**, 26 de março de 2024. Disponível em: <https://blog.esrij.com/2018/11/28/post-31925/>. Acesso em: 21 nov. 2024.

ARVAI, Kevin. K-Means Clustering in Python: A Practical Guide. **Real Python**, 2024. Disponível em: <https://realpython.com/k-means-clustering-python/>. Acesso em: 21 nov. 2024.

GÉRON, Aurélien. **Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow**. O'Reilly Media, Inc., 2022.

GOMES, Pedro César Tebaldi. Clustering: O que é Cluster Analysis e quais suas Aplicações? **Data Geeks**, 09 de junho de 2024.<https://www.datageeks.com.br/clustering/>. Acesso em: 21 nov. 2024.

HUDYMA, Natallia. Jogo de bandeiras do mundo. **Dreamstime**, 2024. Disponível em: <https://pt.dreamstime.com/foto-de-stock-royalty-free-jogo-de-bandeiras-do-mundo-image9201735>. Acesso em: 21 nov. 2024.

IMSUMAND18. Different types of Distances used in Machine Learning. **Medium**, 23 de junho de 2020. Disponível em: <https://medium.com/@imsumand18/different-types-of-distances-used-in-machine-learning-c7cde81b9e03>. Acesso em: 21 nov. 2024.

IZBICKI, Rafael; DOS SANTOS, Tiago Mendonça. **Aprendizado de máquina: uma abordagem estatística**. Rafael Izbicki, 2020.

MÜLLER, Vincent C. **Ethics of artificial intelligence and robotics**. 2020.

PROVOST, F. **Data Science for Business:** What you need to know about data mining and data-analytic thinking. O'Reilly Media, Inc, 2013.

RAJ, Anand Raj. Clustering unveiled: The Intersection of Data Mining, Unsupervised Learning, and Machine Learning. **Medium**, 22 de janeiro de 2024. Disponível em: <https://pub.towardsai.net/clustering-unveiled-the-intersection-of-data-mining-unsupervised-learning-and-machine-learning-a8c9feefb46f>. Acesso em: 21 nov. 2024.

RIYAAGGARWAL. Determinando o número de clusters em mineração de dados. **Acervo Lima**, 2022. Disponível em: <https://acervolima.com/determinando-o-numero-de-clusters-em-mineracao-de-dados/>. Acesso em: 21 nov. 2024.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SANTOS, Diego. Análise de Componentes Principais. **LinkedIn**, 10 de fevereiro de 2024. Disponível em: <https://www.linkedin.com/pulse/an%C3%A1lise-de-componentes-principais-diogo-santos-xxqpf/>. Acesso em: 21 nov. 2024.

SANTOS, Vinicius. Como funciona o algoritmo de Agrupamento Hierárquico. **Computer Science Masters**, 28 de fevereiro de 2019. Disponível em: <https://www.computersciencemaster.com.br/como-funciona-o-algoritmo-de-agrupamento-hierarquico/>. Acesso em: 21 nov. 2024.

SENGILEY. Anomaly detection based on clustering. **Cross Validated**, 06 de dezembro de 2016. Disponível em: <https://stats.stackexchange.com/questions/160260/anomaly-detection-based-on-clustering>. Acesso em: 21 nov. 2024.

SILVA, Tarcízio. Linha do Tempo do Racismo Algorítmico: casos, dados e reações. **Tarcízio Silva**. 2024. Disponível em: <https://tarciziosilva.com.br/blog/destaques/posts/racismo-algoritmico-linha-do-tempo/>. Acesso em: 21 nov. 2024.

SURESH, Harini; GUTTAG, John. A framework for understanding sources of harm throughout the machine learning life cycle. *In*: Proceedings of the 1st ACM Conference on Equity and Access in Algorithms, Mechanisms, and Optimization. p. 1-9. **ACM Digital Library**, 04 de novembro de 2021. Disponível em: <https://dl.acm.org/doi/abs/10.1145/3465416.3483305>. Acesso em: 21 nov. 2024.

YAZDANI, Hamid Reza; Shojaeifard, Alireza. Hybrid Completely Positive Tensorial PCA method for face recognition. **ResearchGate**, 2021. Disponível em: <https://www.researchgate.net/figure/The-scheme-of-PCA-method_fig1_353902821>. Acesso em: 21 nov. 2024.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
DBSCAN	Density-Based Spatial Clustering of Applications with Noise.
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise.
PCA	Análise de Componentes Principais.
SVD	Singular Value Decomposition.
WCSS	Within-Cluster Sum of Squares.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com